Practical No:6

9.Interface Pi Camera with Raspberry Pi to capture image and video.

A]Raspberry pi-kit

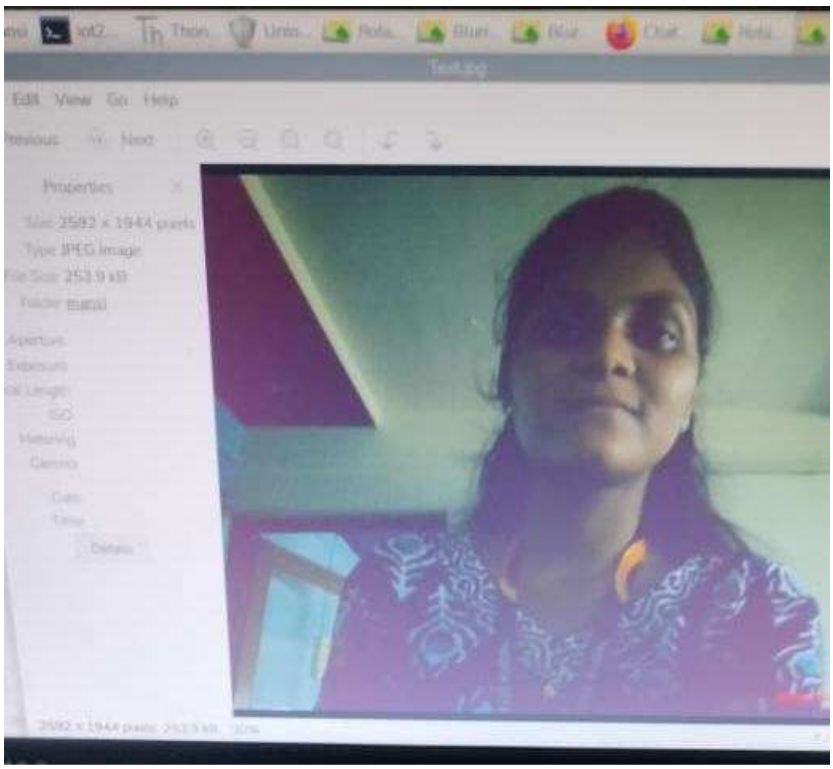
i)Capture an image:

Code:

```
from picamera2 import Picamera2
import time
picam2 = Picamera2()
config = picam2.create_still_configuration()
picam2.configure(cofig)
picam2.start()
time.sleep(2)
picam2.capture_file("image.jpg")
picam2.stop()
print("Image Captured Successfully")
```

Output:

Image Capture Successfully



ii)Capture an video:

Code:

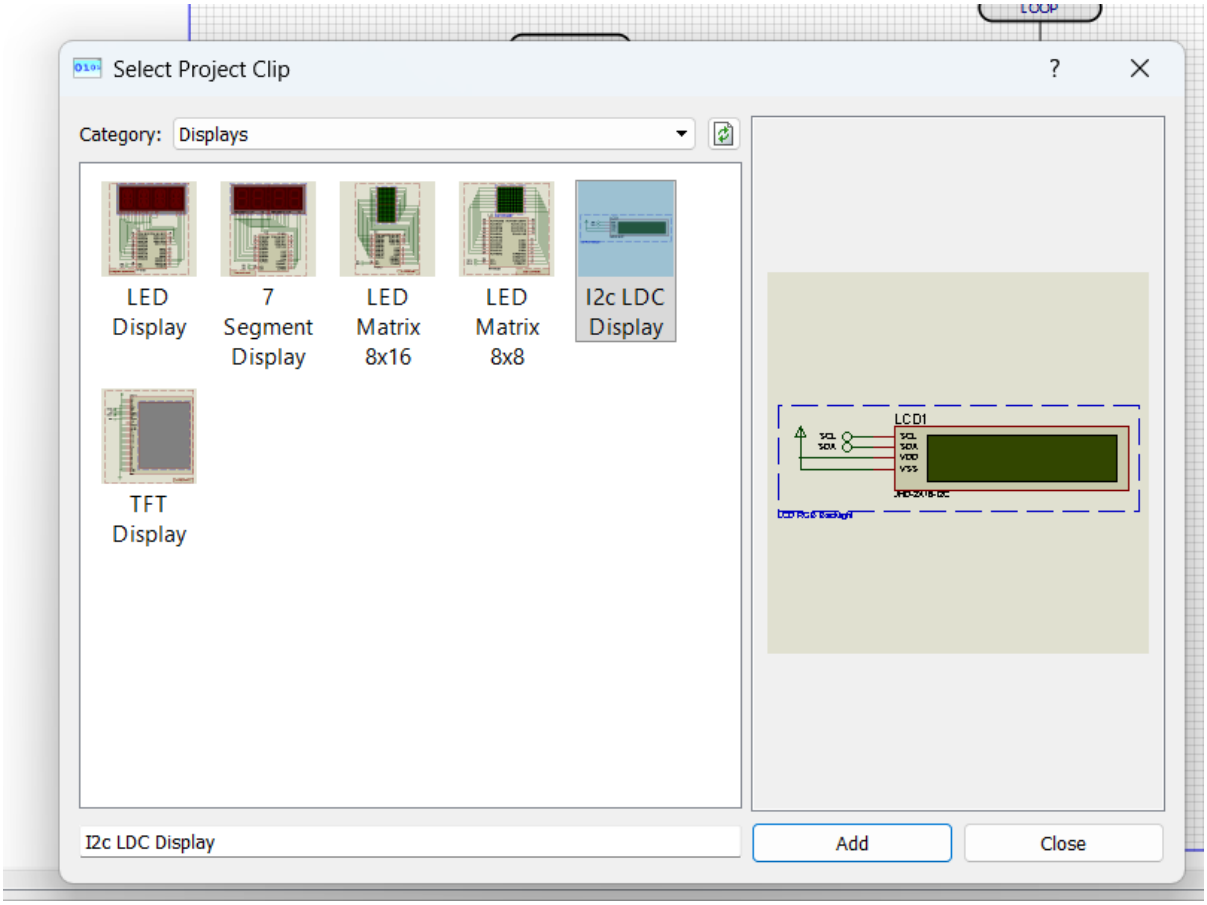
```
from picamera2 import Picamera2
from picamera2.encoders import H264Encoder
import time
picam2 = Picamera2()
video_config = picam2.create_video_configuration()
picam2.configure(video_config)
encoder = H264Encoder()
picam2.start()
picam2.start_recording(encoder, "video.h264")
time.sleep(10)
picam2.stop_recording()
picam2.stop()
print("Video Recorded Successfully")
```

Output:

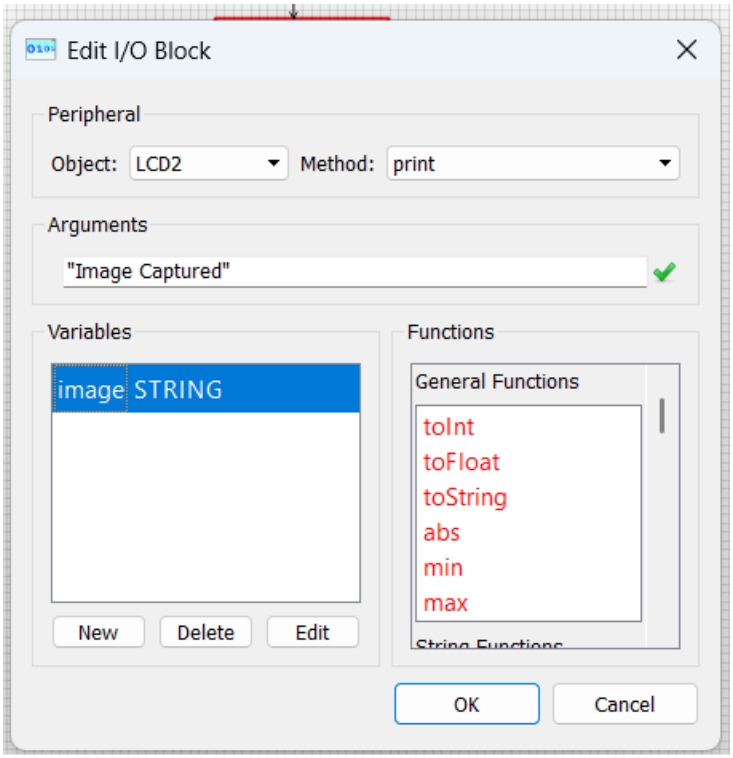
Video Record Successfully

B]Simulation using 16x2

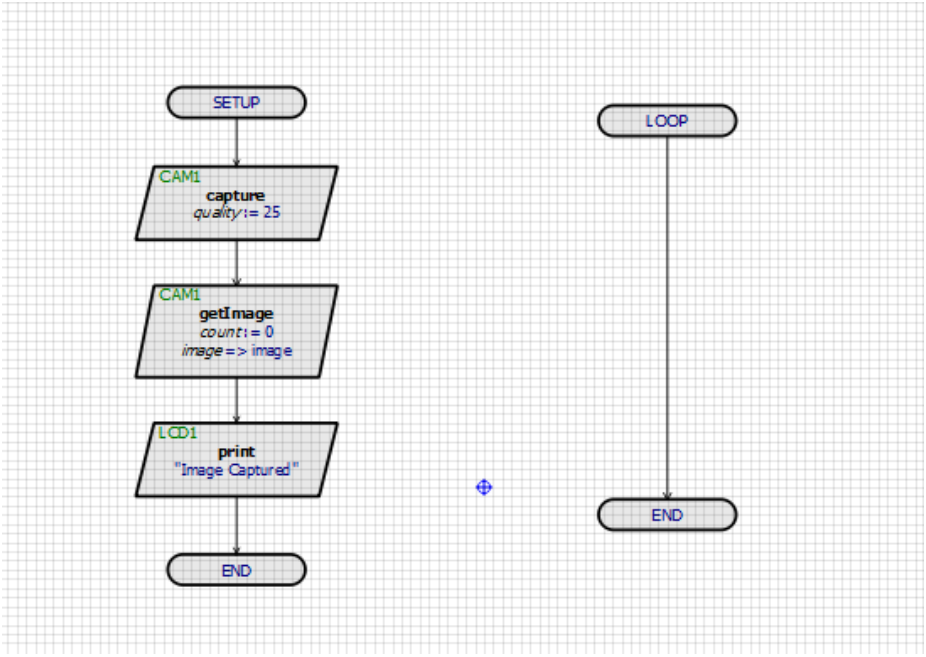
1]Select i2c Display to get further precised output



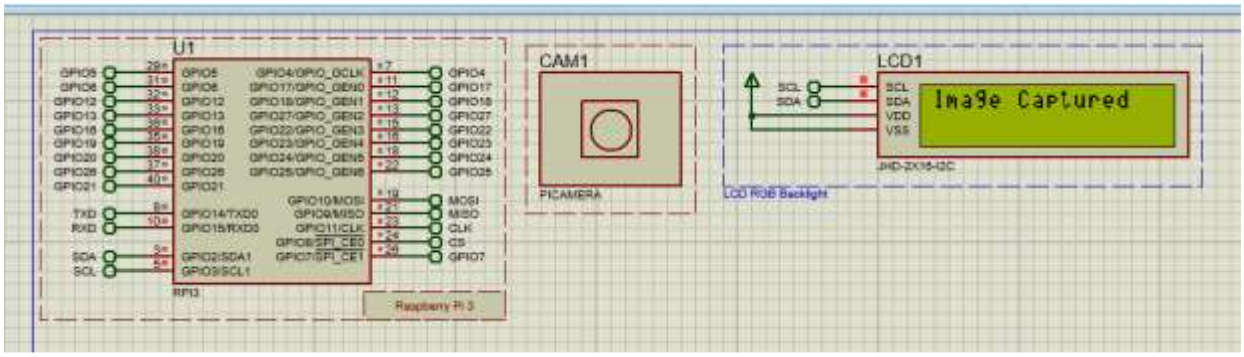
2]Write printing statement in 16x2 LCD



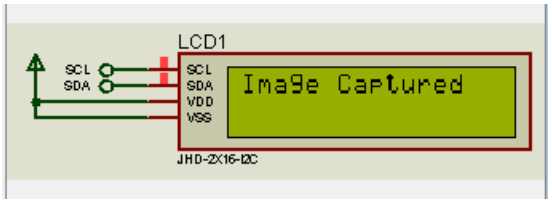
3]Create workflow of capture image



4]Raspberry pi -connect with camera model and 16x2 LCD

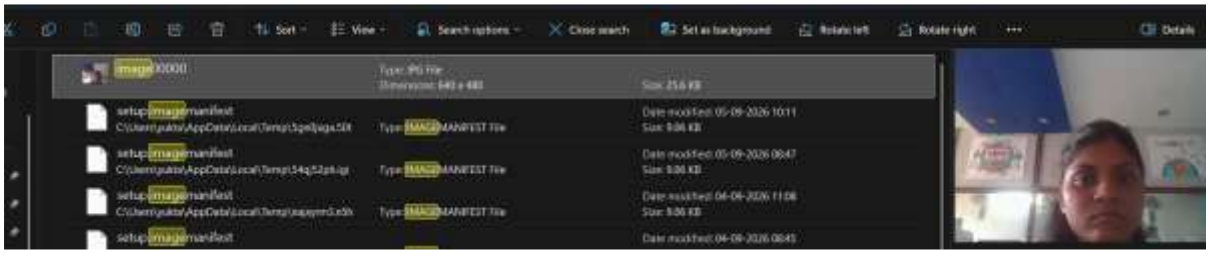


5]Output in visual designer



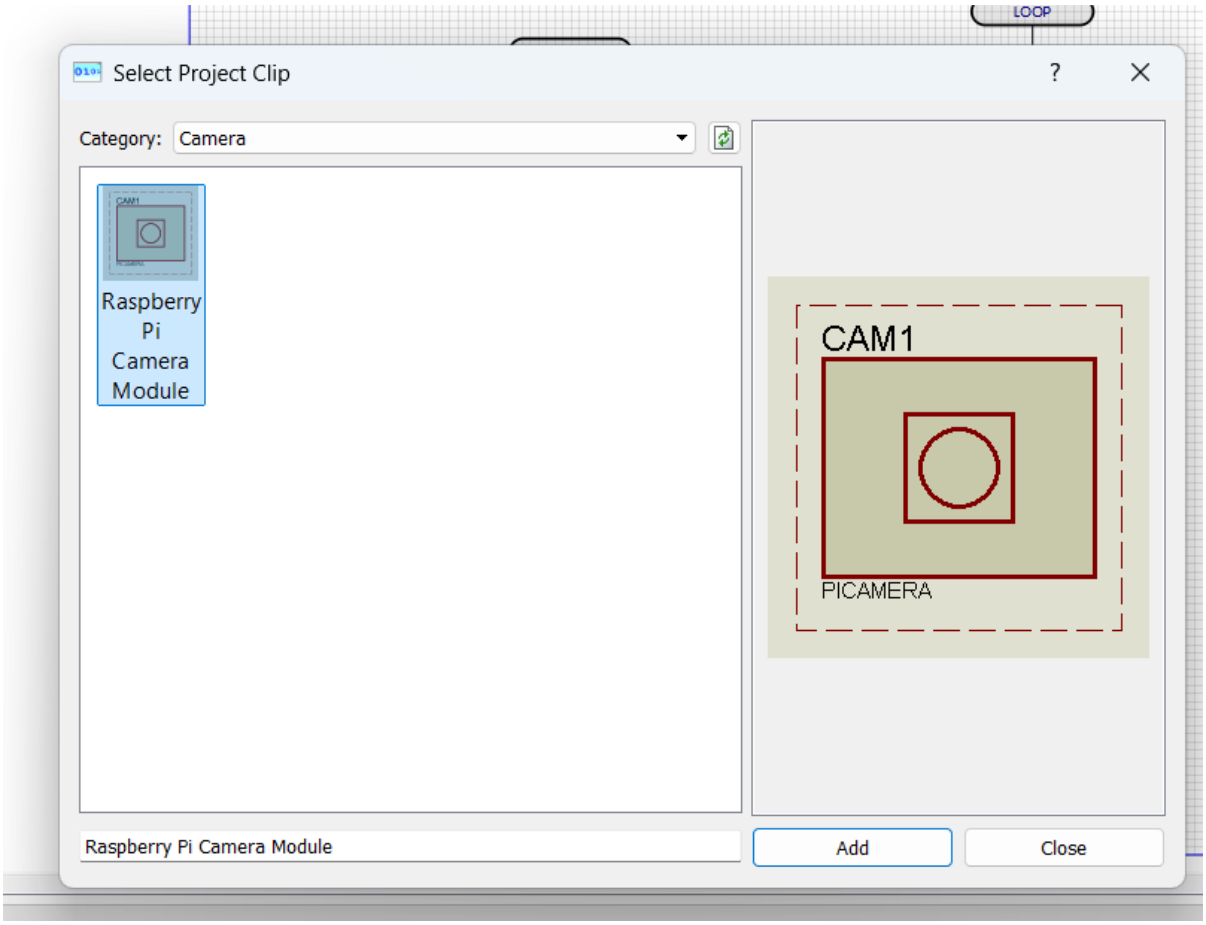
6]Captured Image and location

C:\Users\yukta\AppData\Local\Temp

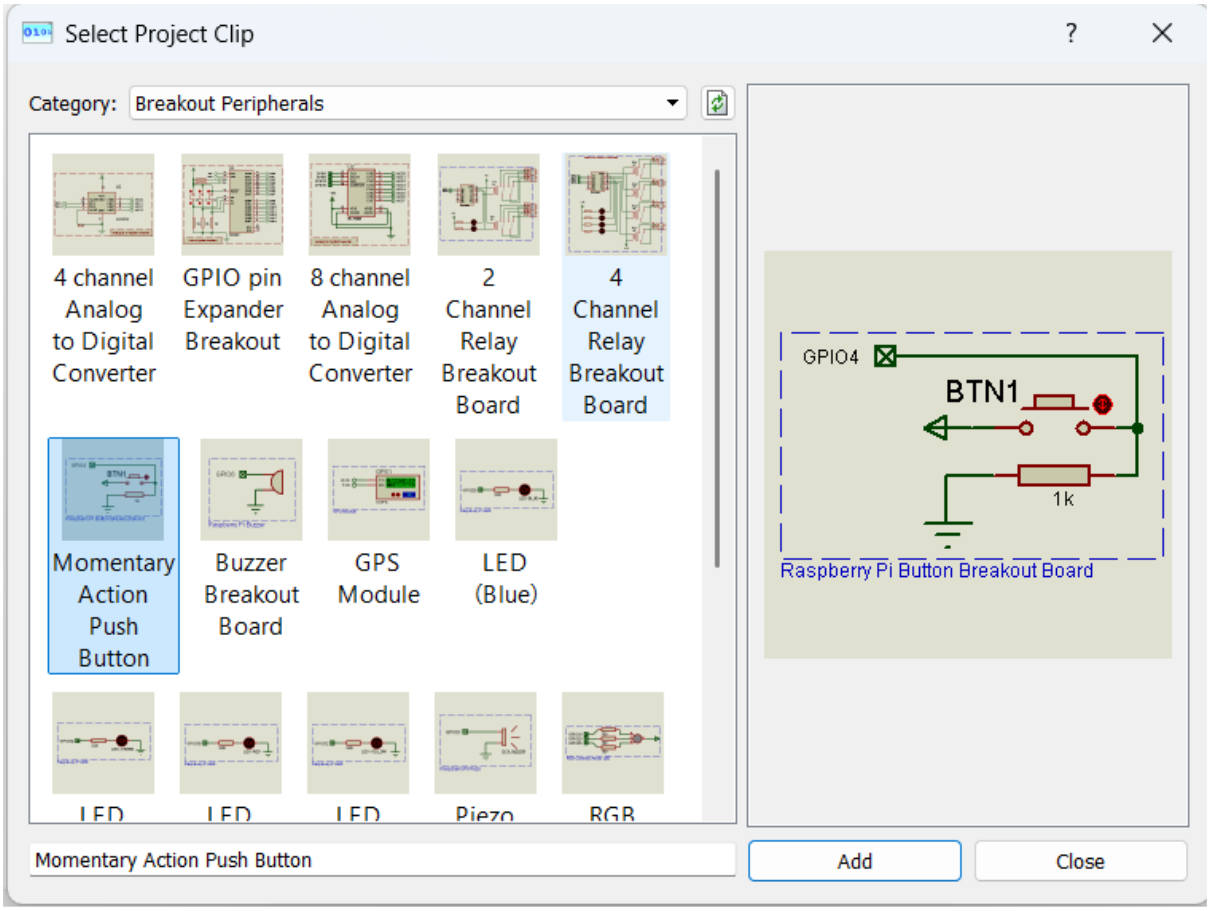


C]Simulation using camera module

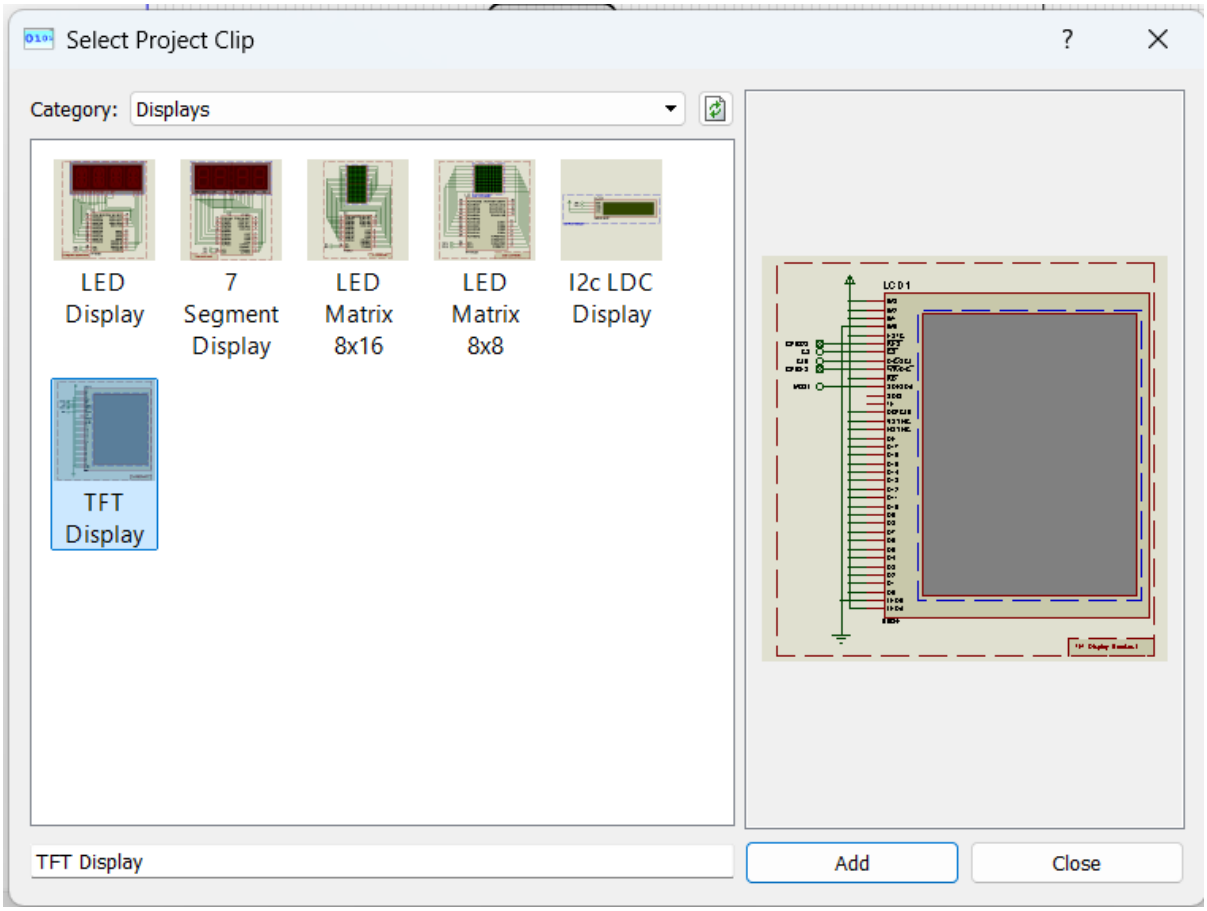
1]Selecting camera module by adding peripherals



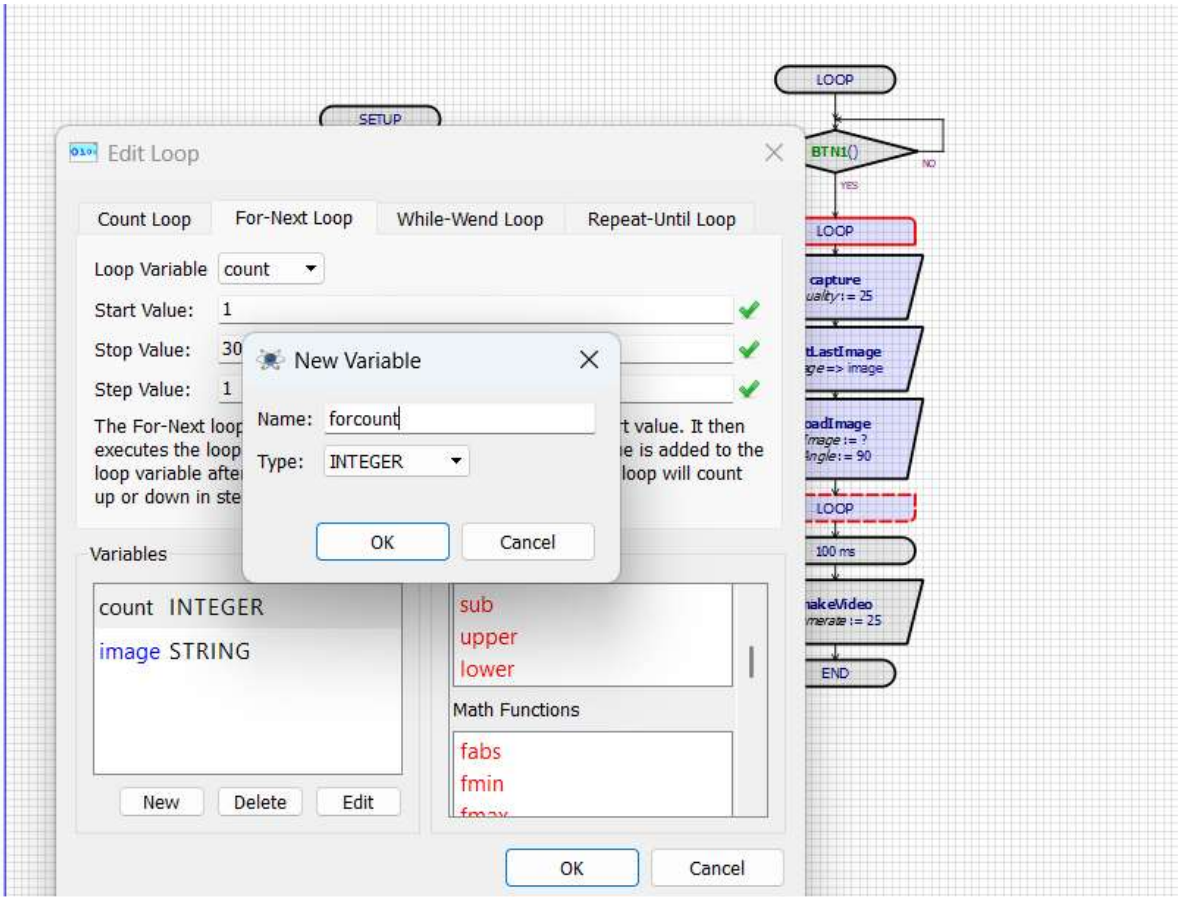
2]Selecting Push button to start camera from breakout peripherals.



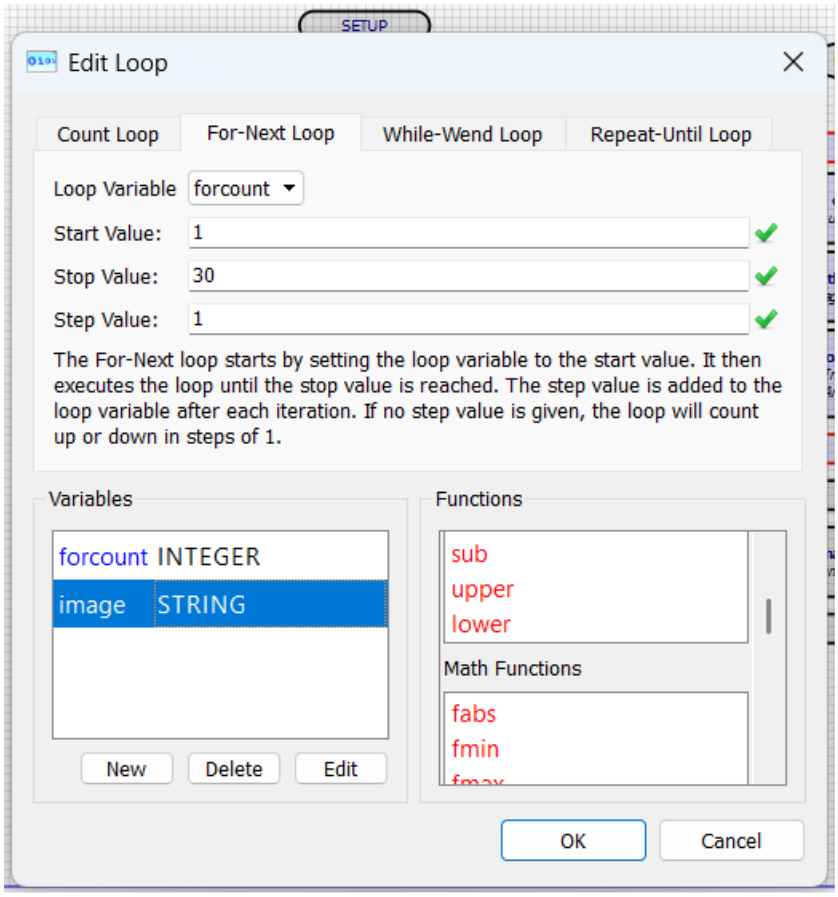
3]Selecting TFT Display to display recording image from Displays



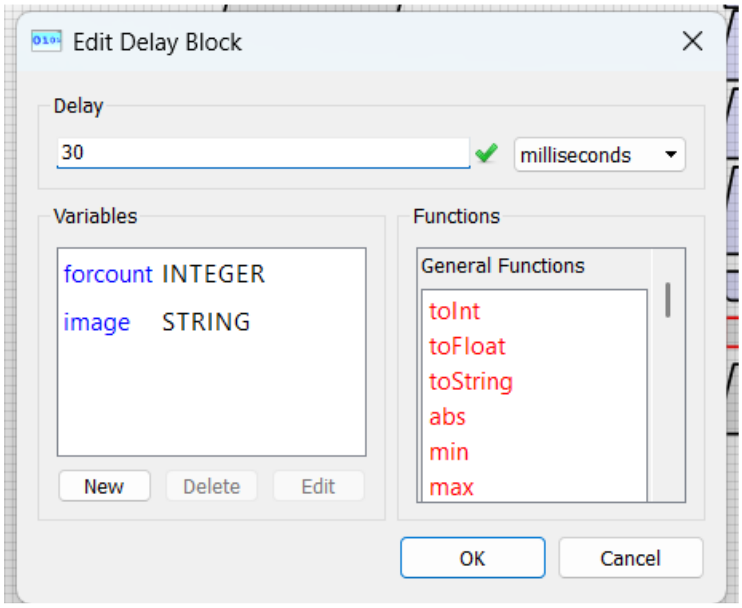
4]Selecting variable forcount because of for-next loop execution



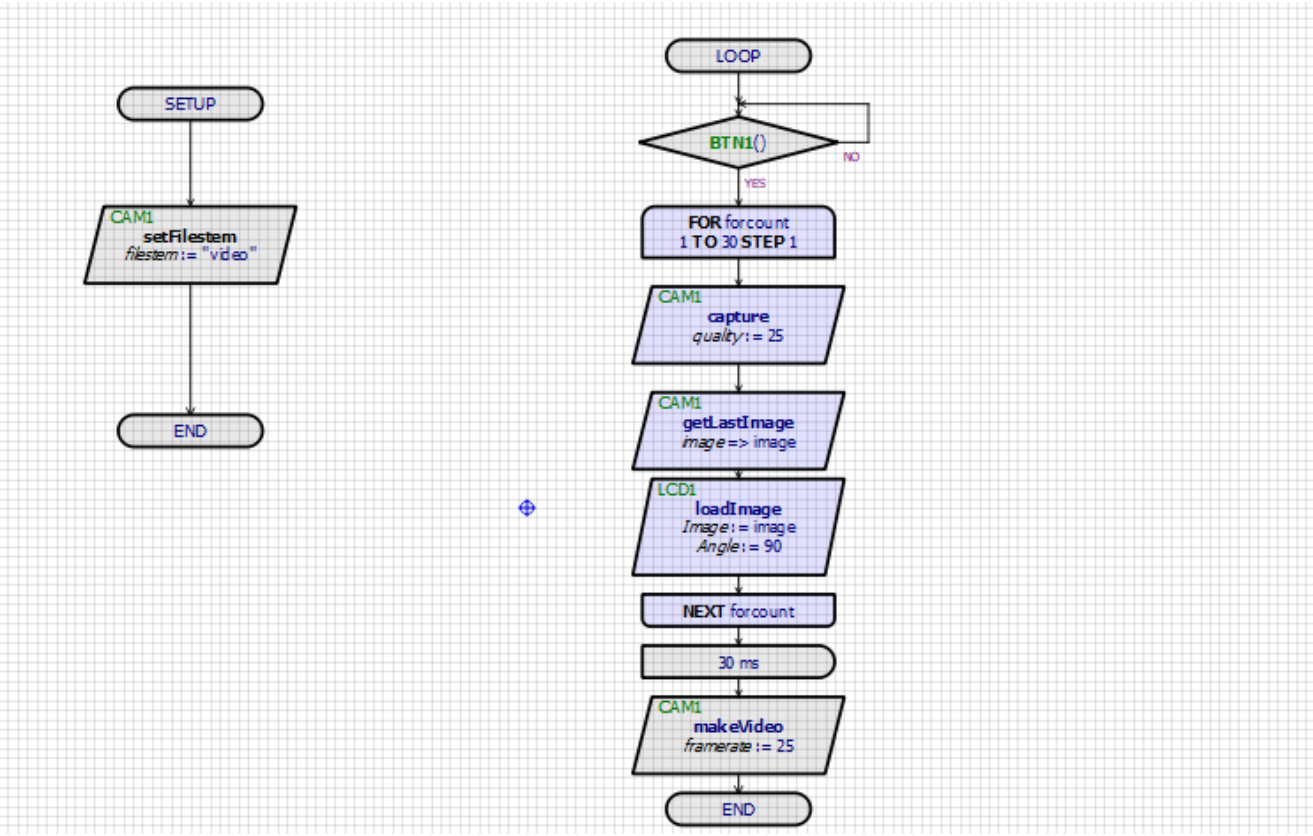
5]Assigning value to forcount



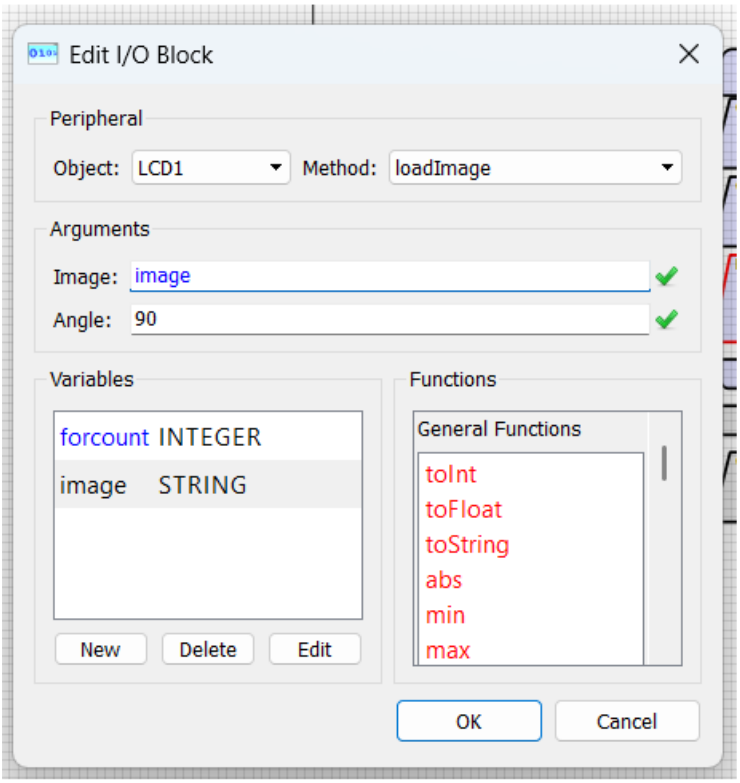
6]Assigning value to Delay block



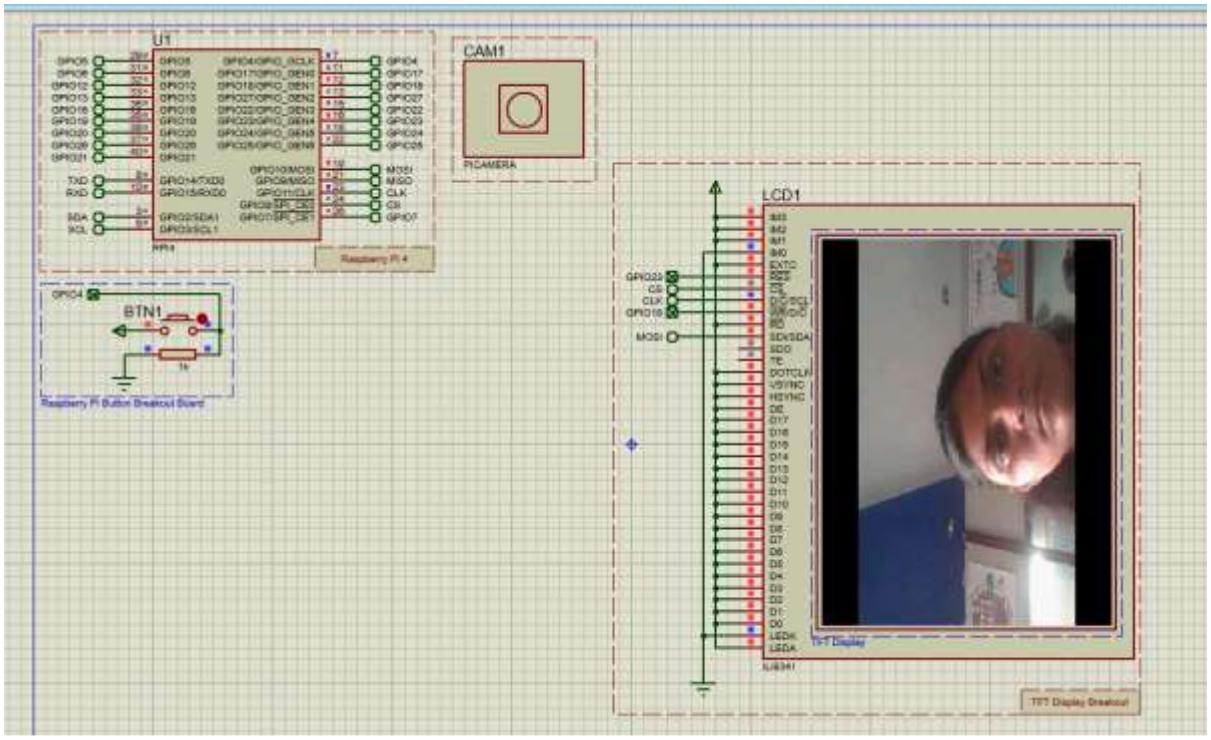
7]Flowchart of the working structure of camera simulation



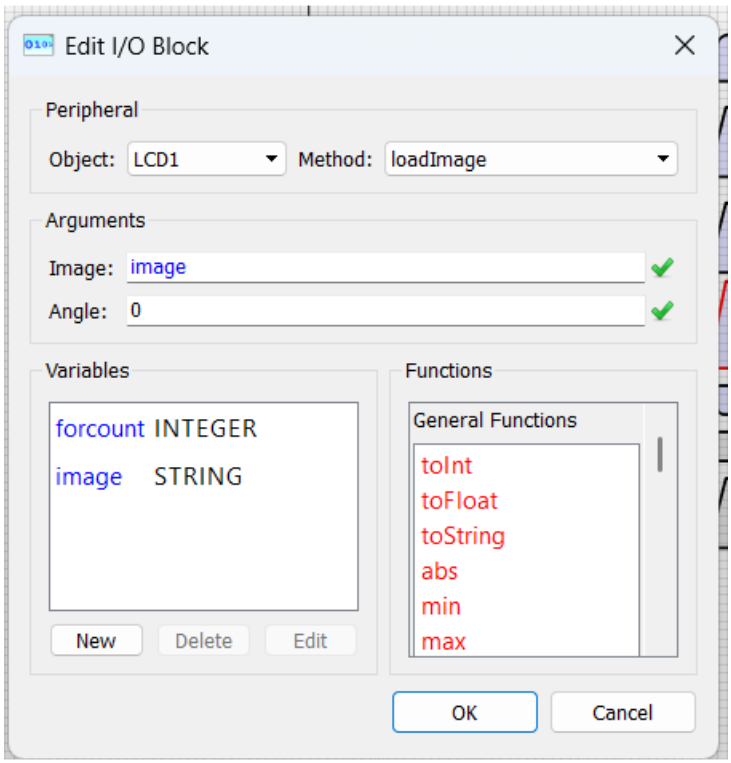
8]Assinging angle to 90 Degree for rotating recording image



9]Output of Rotated 90 Degere Image



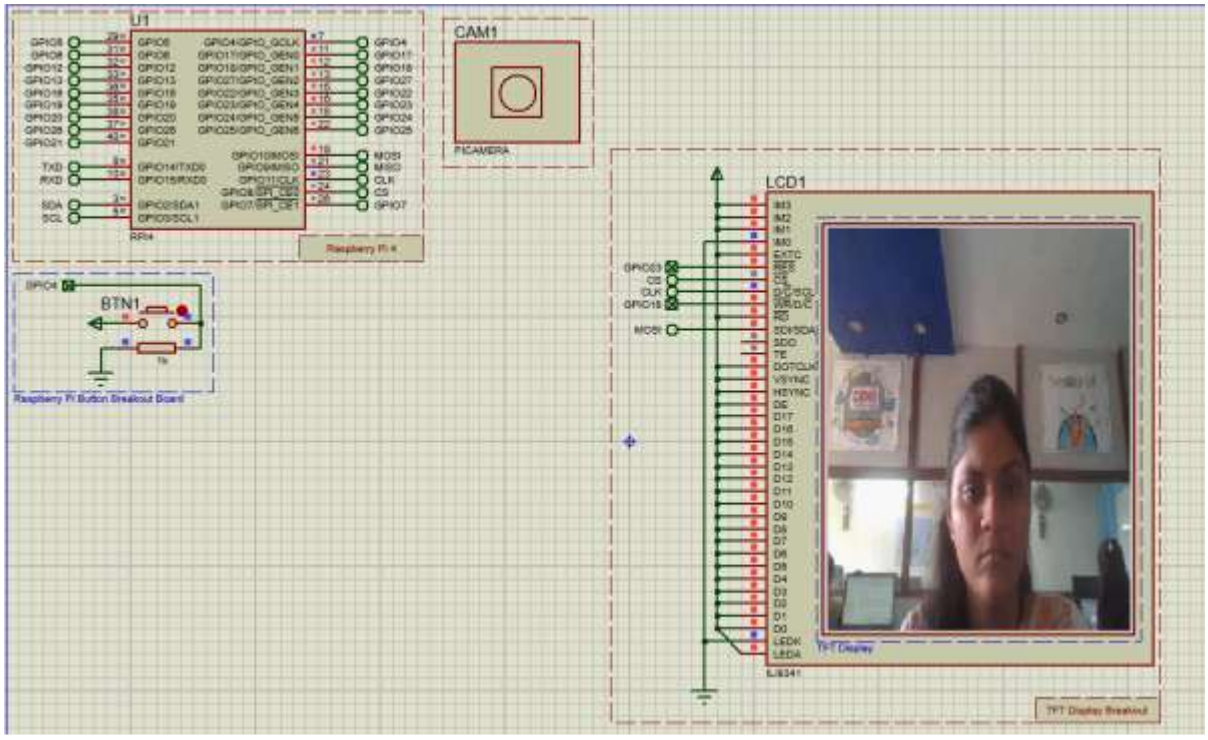
10] Assinging angle to 0 Degree for stable recording image



11]Output in Visual Designer of stable image



12]Output of stable image in Schematic capture



13]Recorded video location

C:\Users\yukta\AppData\Local\Temp



PRACTICAL NO 7

14. Design a web dashboard using Flask. Control relay module via web. Code:

Index.html :

```
<!DOCTYPE html>
<html>
<head>
<title>Raspberry Pi Control</title>
</head>
<body>
<h1>Raspberry Pi Control</h1>
<button onclick="controlLED('on')">LED ON</button>
<button onclick="controlLED('off')">LED OFF</button>
```

```
<script>
function controlLED(state) { fetch("/led/" + state)
.then(response => response.json())
.then(data => alert(data.status));
}
</script>
</body>
</html>
```

App.py :

```
from flask import Flask, render_template, redirect, url_for from gpiozero import LED
app = Flask( name )
# LED GPIO 17 ला connected आहे
led = LED(17)
@app.route("/") def home():
return render_template("index.html", led_status=led.is_lit)
@app.route("/led/on") def led_on():
led.on()
return redirect(url_for("home"))
@app.route("/led/off") def led_off():
led.off()
return redirect(url_for("home"))
if name == " main ": app.run(host="0.0.0.0", port=5000)
```

output :



A web dashboard was successfully designed using the Flask framework in Python to control a relay module connected to a Raspberry Pi.

The hardware circuit was integrated with the web backend using GPIO pins.

Pressing the "LED ON" and "LED OFF" buttons on the web interface successfully activated and deactivated the relay module to control the bulb status in real-time.

PRACTICAL NO 8

18. Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi.
Code :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

<title>Raspberry Pi Monitor</title>

</head>

<body>

<h1>Raspberry Pi 5</h1>

<h2>CPU: {{ cpu }} %</h2>

<h2>Memory: {{ memory }} %</h2>

<h2>Temperature: {{ temp }} °C</h2>

</body>

</html>
```

App.py :

```
from flask import Flask, render_template import os

app = Flask( name ) @app.route("/")

def home():

# CPU Temperature

with open("/sys/class/thermal/thermal_zone0/temp") as f: temp = int(f.read()) / 1000

# Memory

with open("/proc/meminfo") as f: data = f.readlines()

total = int(data[0].split()[1])

free = int(data[2].split()[1]) memory = (total - free) * 100 / total # CPU Load

load = os.getloadavg()[0]

cpu = load * 100 / os.cpu_count() return render_template(

"index.html", cpu=round(cpu, 1),

memory=round(memory, 1),
```

```
temp=round(temp, 1)
)
app.run(host="0.0.0.0", port=5000)
```

output:



Practical No : 1

1 . Create different blinking patterns with LEDs by connecting to GPIO pints of Raspberry Pi.

A .Using Raspberry pi 3

Output:

The 4 LEDs blink one after another with a 0.5-second delay, continuously repeating the sequence.

B . Using Proteus 8 Professional

Step 1: Open Proteus 8 Professional and create a new project.

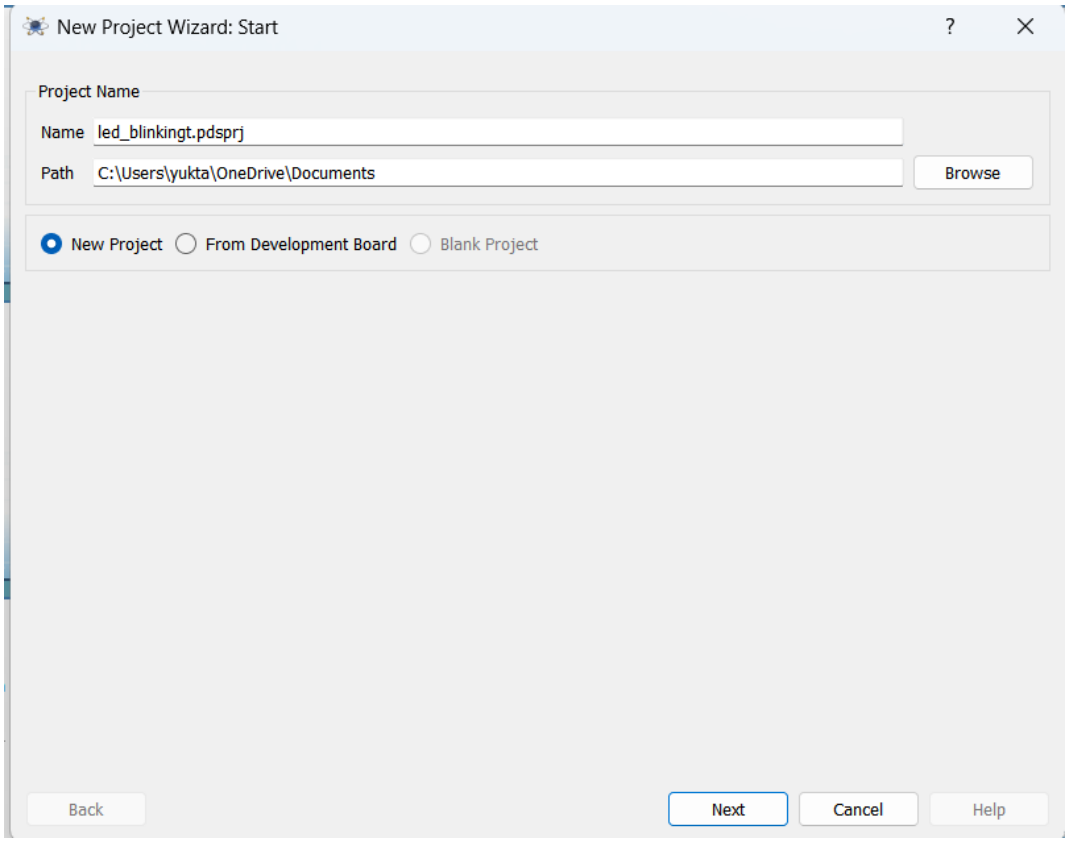


Figure 1.1: create the new project

Step 2: Created a Flowchart Project for Raspberry Pi 4

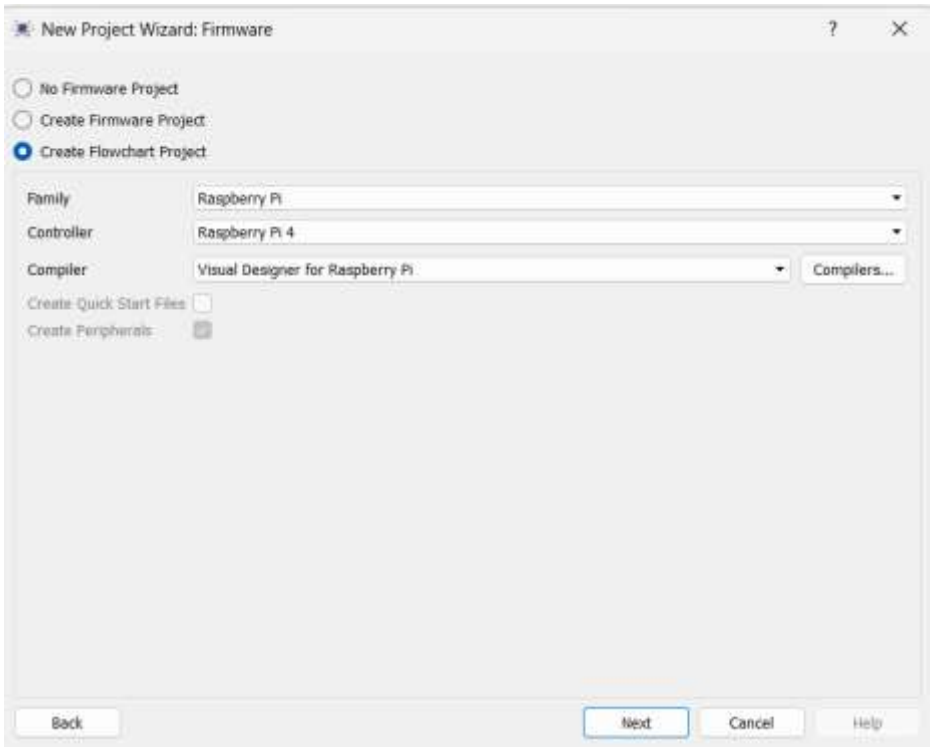


Figure 1.2: project to LED component

Step 3: to selected the LED component from the Pick Devices window

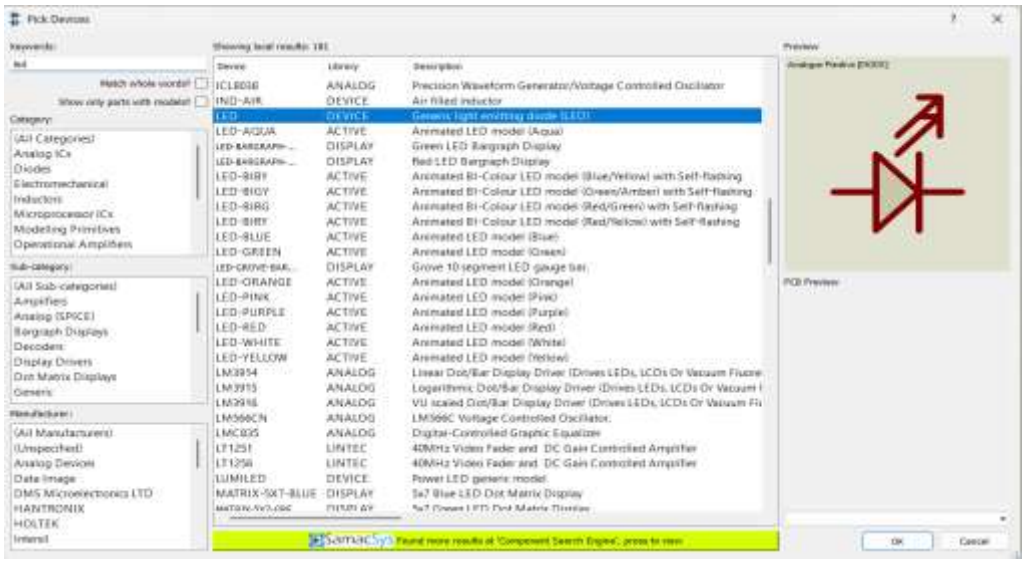


Figure 1.3: Choosing Appropriate Devices

Step 4 : code:

```
import RPi.GPIO as GPIO

import time

GPIO.setmode(GPIO.BCM)

LED1 = 17

LED2 = 18

LED3 = 27

LED4 = 22

GPIO.setup(LED1, GPIO.OUT)

GPIO.setup(LED2, GPIO.OUT)
```

```
GPIO.setup(LED3, GPIO.OUT)
GPIO.setup(LED4, GPIO.OUT)
try:
while True:
# Pattern 1: One by one
GPIO.output(LED1, GPIO.HIGH)
time.sleep(1)
GPIO.output(LED1, GPIO.LOW)

GPIO.output(LED2, GPIO.HIGH)
time.sleep(1)
GPIO.output(LED2, GPIO.LOW)

GPIO.output(LED3, GPIO.HIGH)
time.sleep(1)
GPIO.output(LED3, GPIO.LOW)

GPIO.output(LED4, GPIO.HIGH)
time.sleep(1)
GPIO.output(LED4, GPIO.LOW)


# Pattern 2: All LEDs ON
GPIO.output(LED1, GPIO.HIGH)
```

```
GPIO.output(LED2, GPIO.HIGH)

GPIO.output(LED3, GPIO.HIGH)

GPIO.output(LED4, GPIO.HIGH)

time.sleep(1)

# All LEDs OFF

GPIO.output(LED1, GPIO.LOW)

GPIO.output(LED2, GPIO.LOW)

GPIO.output(LED3, GPIO.LOW)

GPIO.output(LED4, GPIO.LOW)

        time.sleep(1)

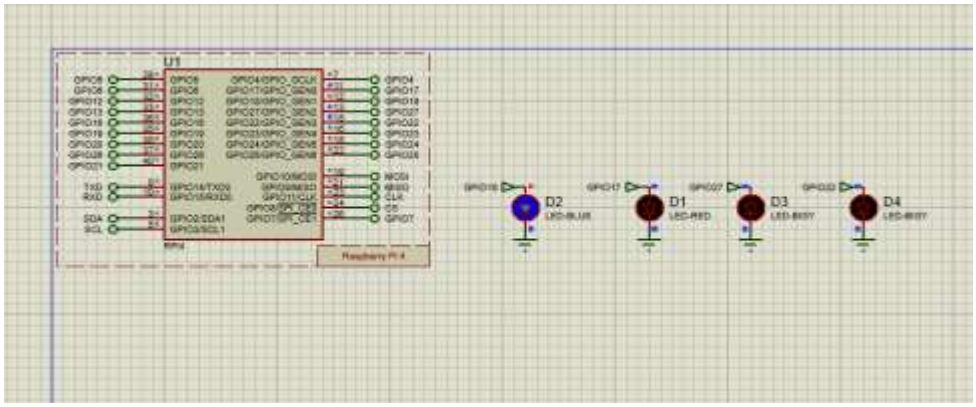
        except KeyboardInterrupt:

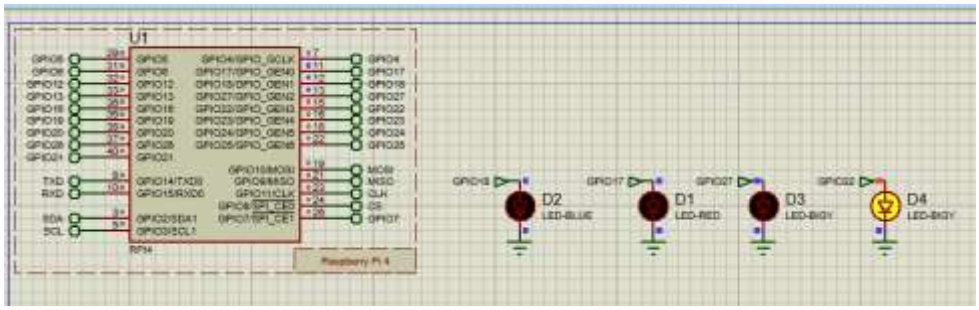
pass

finally:

GPIO.cleanup()
```

Step 3: Connect the selected LED components to the appropriate GPIO pins of Raspberry Pi





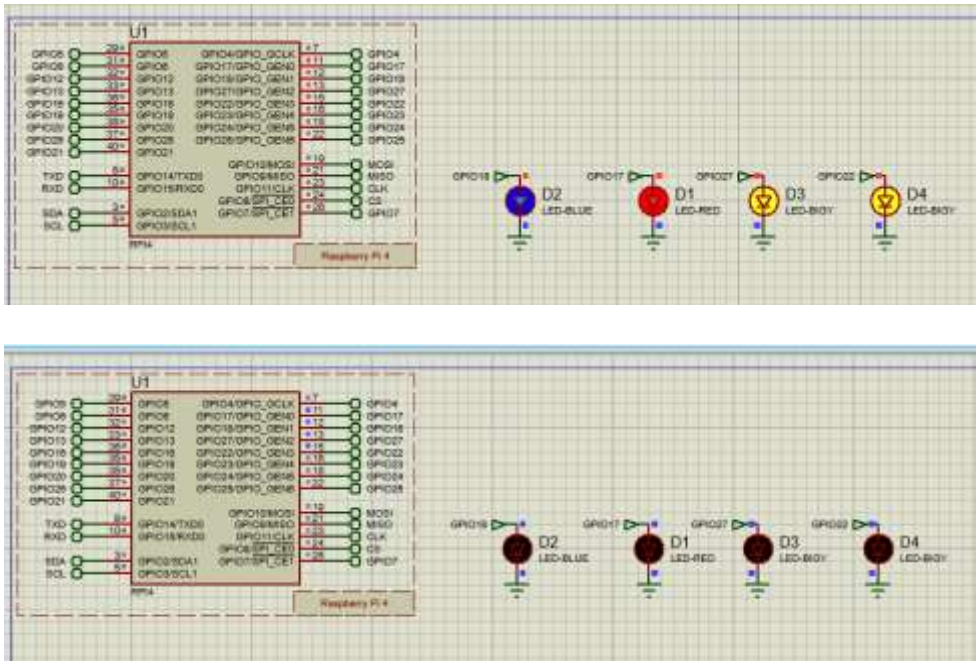


Figure 1.4: Circuit Diagram of Blinking LEDs using Raspberry Pi 4

Practical No:3

3.Interface TM1637 with Raspberry pi and Display current time.

A]Raspberry pi-kit:

Code:

```
import tm1637

from time import sleep

CLK = 24

DIO = 23

display = tm1637.TM1637(clk=CLK, dio=DIO)

display.brightness = 5

while True:

    display.numbers(12, 30)
```

```
sleep(5)

display.numbers(18, 78)

sleep(5)
```

Output:

Display current time in this seven segment

B]Simulation

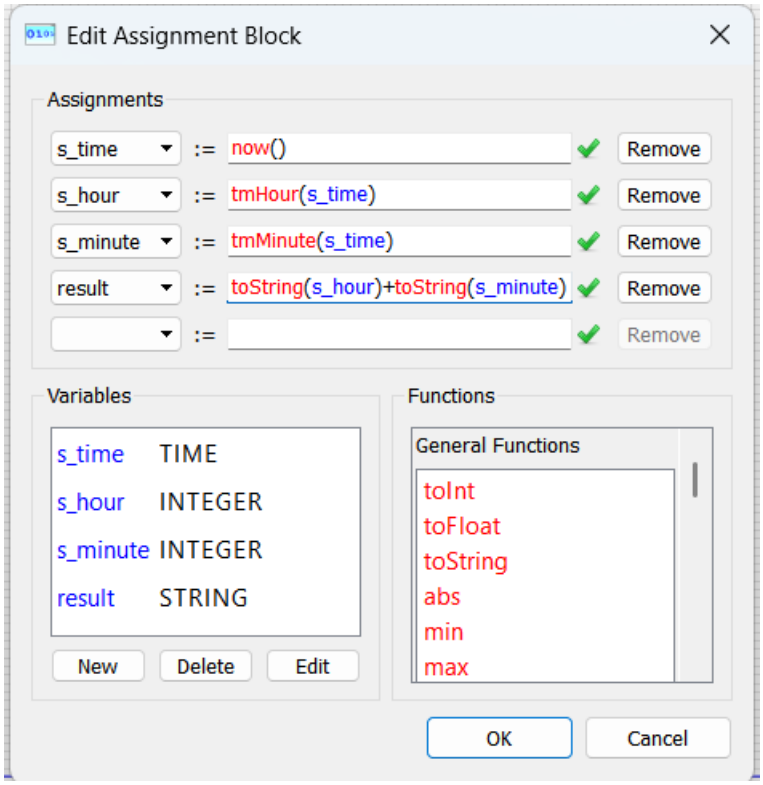


Figure 3.1: Create variables and assign function

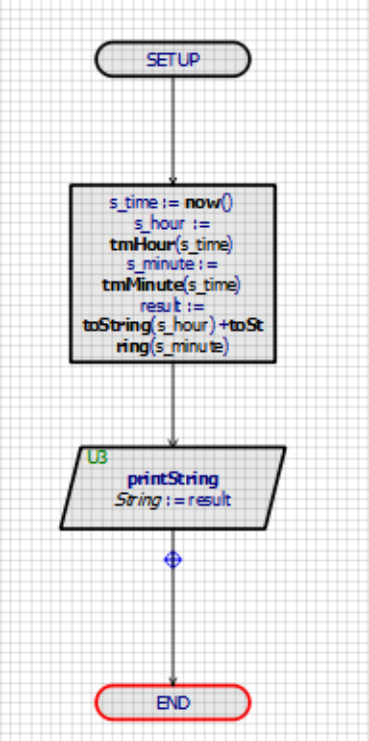


Figure 3.2: Create workflow of seven segment

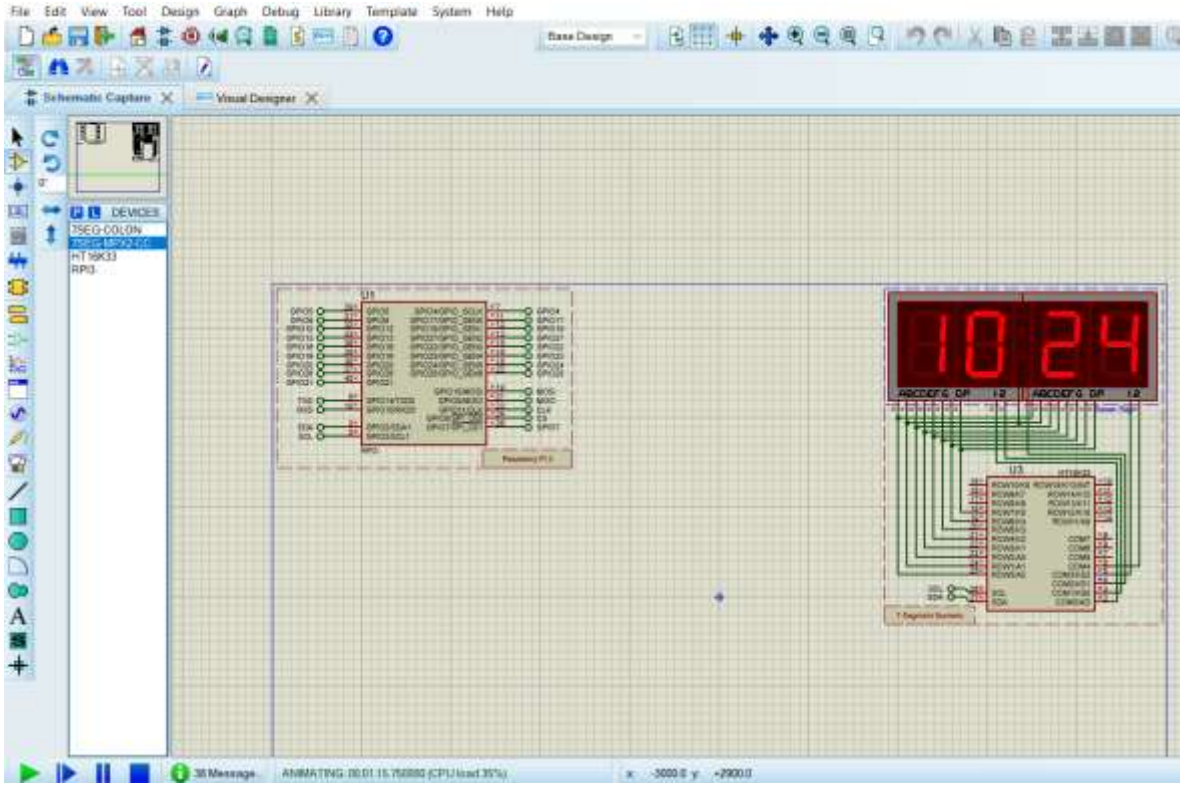


Figure 3.3: Raspberry Pi–Based Digital Clock Simulation Using Proteus



Figure 3.4: Display current time on seven segment

Practical No:4

6. Interface 16x2 LCD with Raspberry Pi using I2C module to display text.

A]Raspberry pi -kit

Code:

```
from RPLCD.i2c import CharLCD
lcd = CharLCD('PCF8574', 0x27)
lcd.write_string("Yukta Koyande")
```

Output:

Display output **"Yukta Koyande"**

B]Simulation

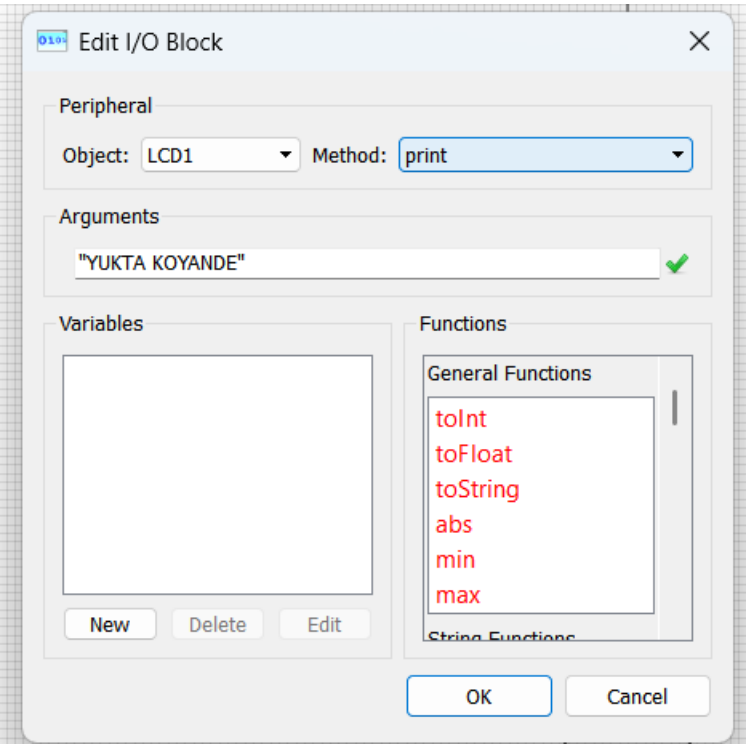


Figure 4.1: Write a display value in 16x2 LCD

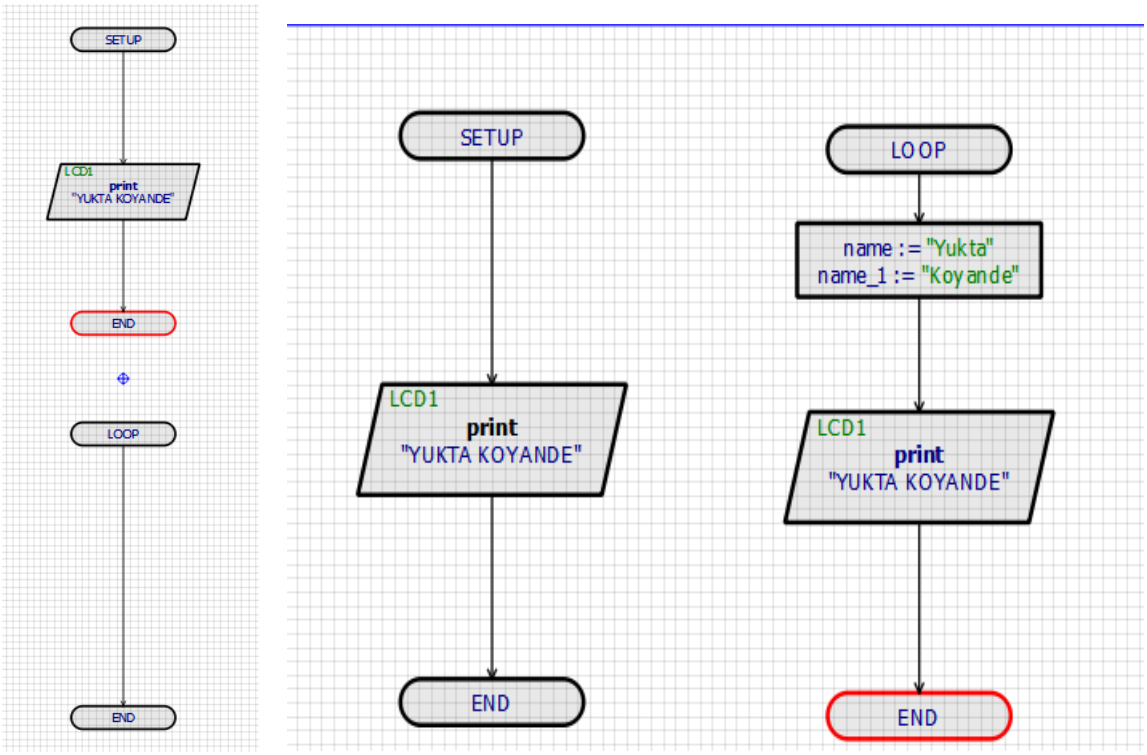


Figure 4.2: Create workflow of 16x2 LCD model

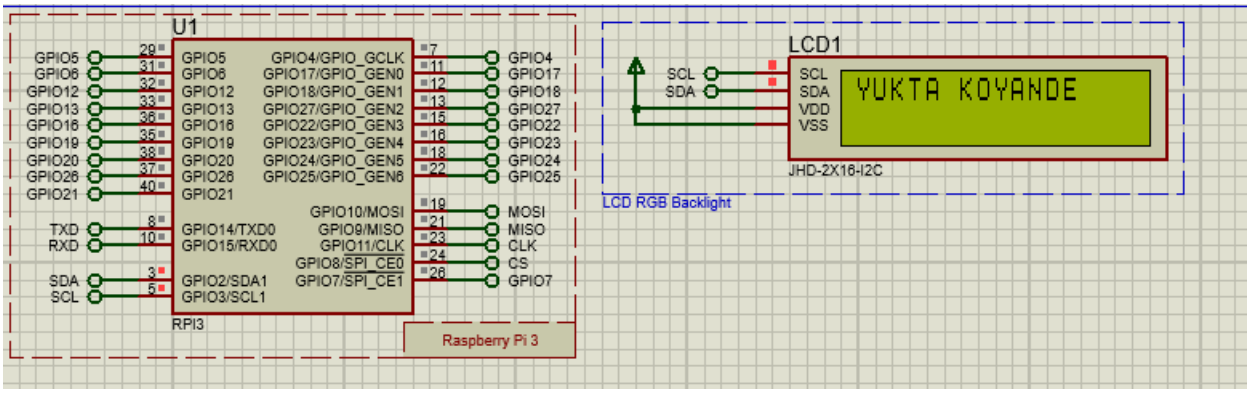


Figure 4.3: Raspberry pi – based value printing in 16x2 LCD using protues

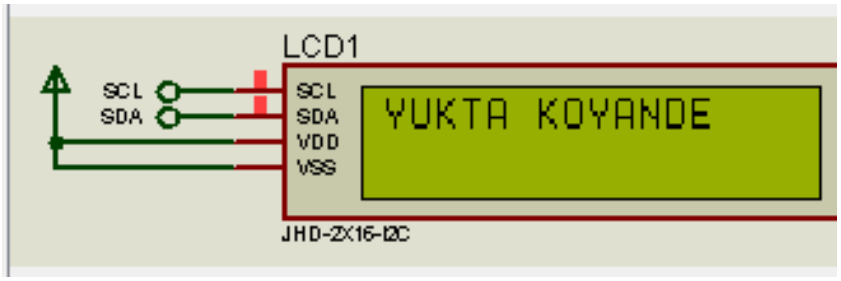


Figure 4.4: Display the output in 16x2 LCD

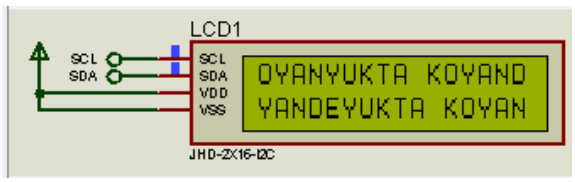


Figure 4.5:Loop Scroll

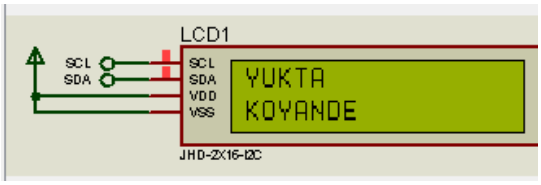


Figure 4.5:Space indendation

ADS SIMULATION

B)Simulation

9. Interface ADS 1115 with Raspberry Pi to act as oscilloscope.

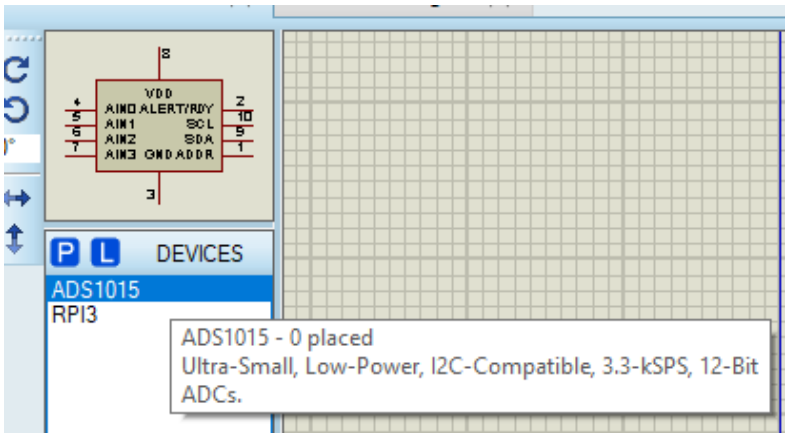


Fig 9.2 Pick ADS

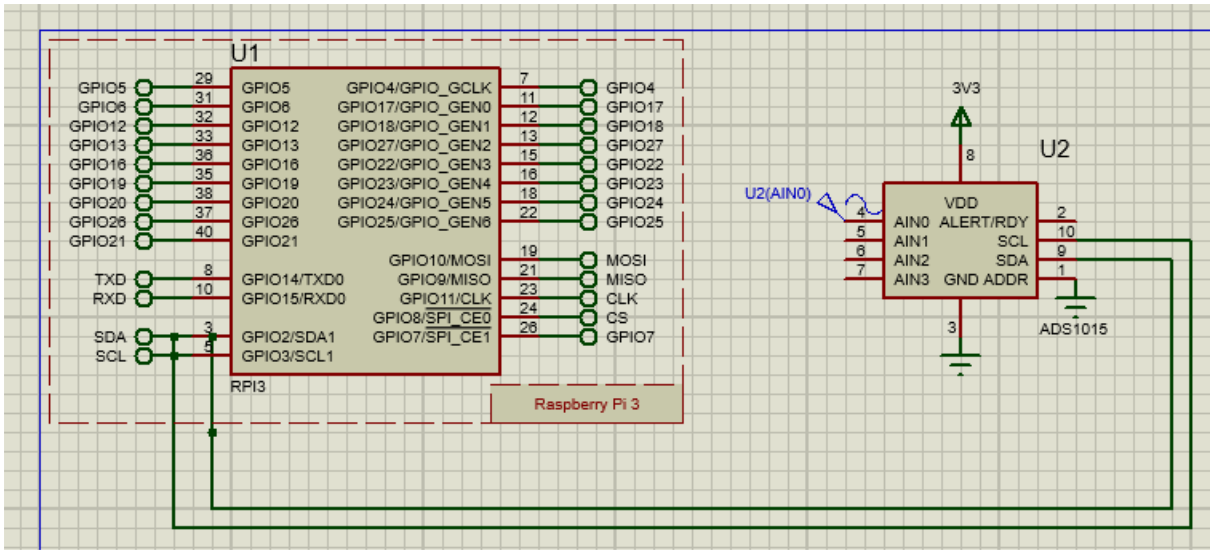


Fig 9.3 Connect the wires to SDA,SCL,ADDR,AINO

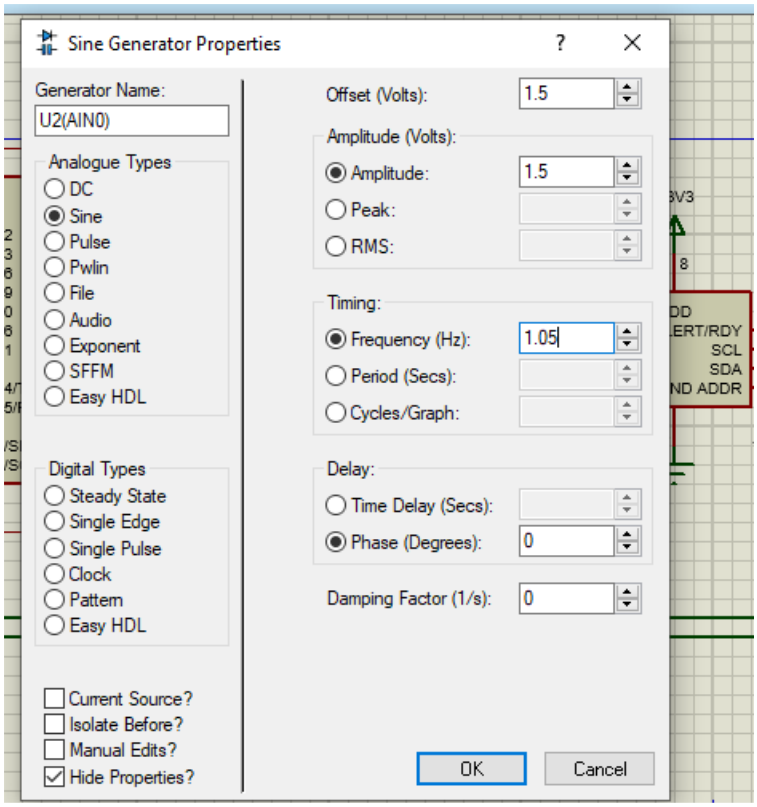


Fig 9.4 Set AINO properties

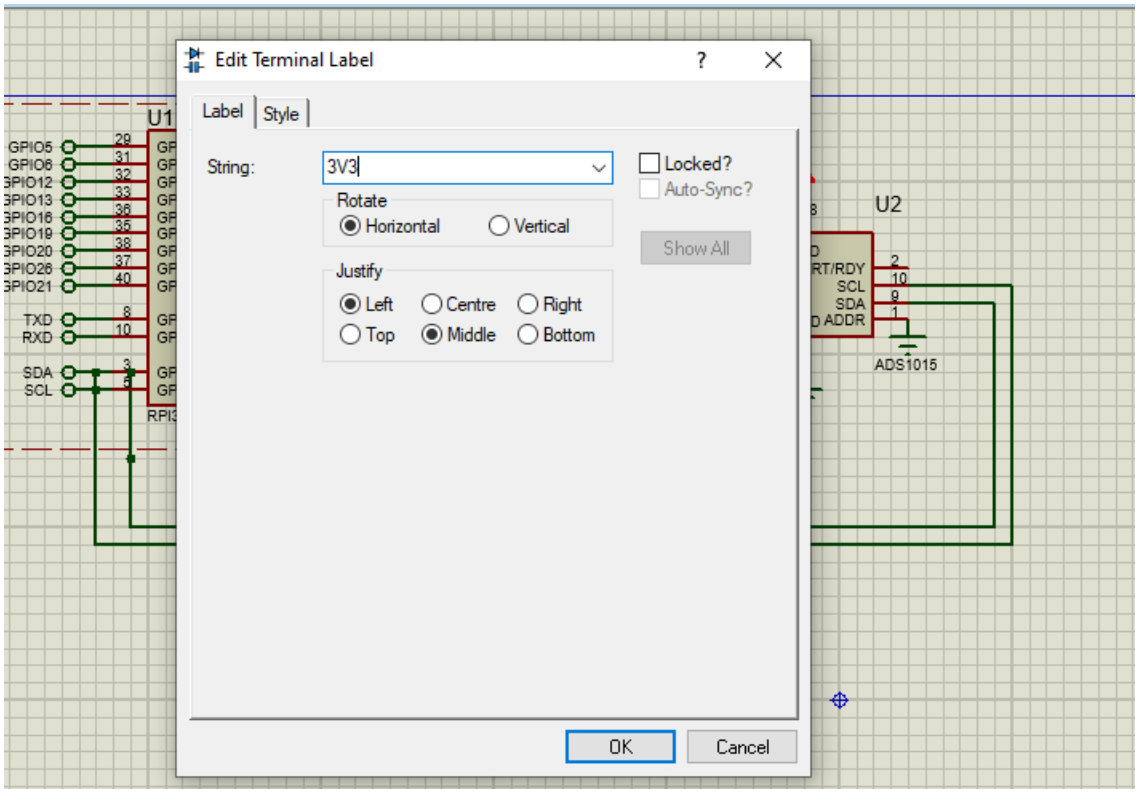


Fig 9.5 Set POWER properties

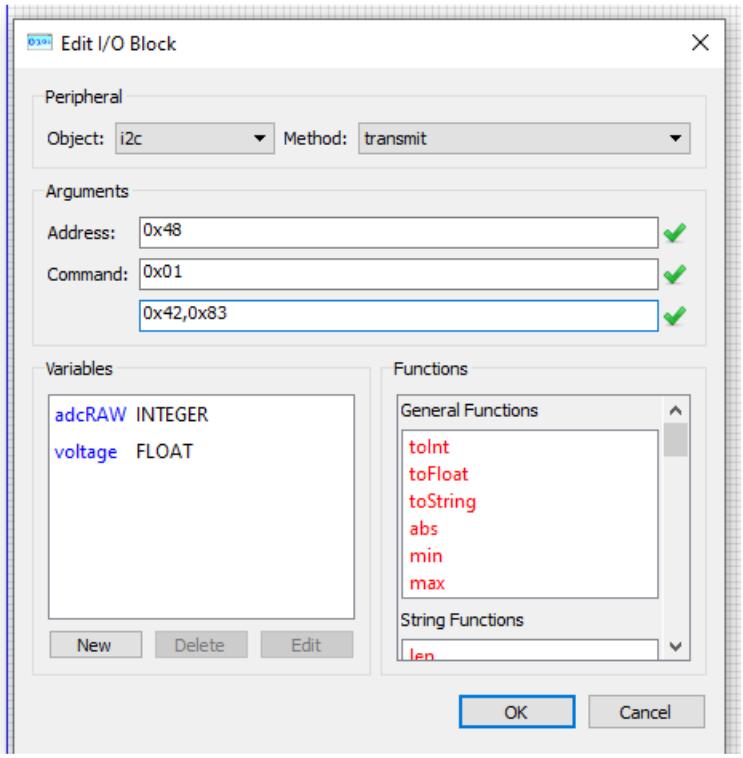


Fig 9.6 Add transmit block and set values

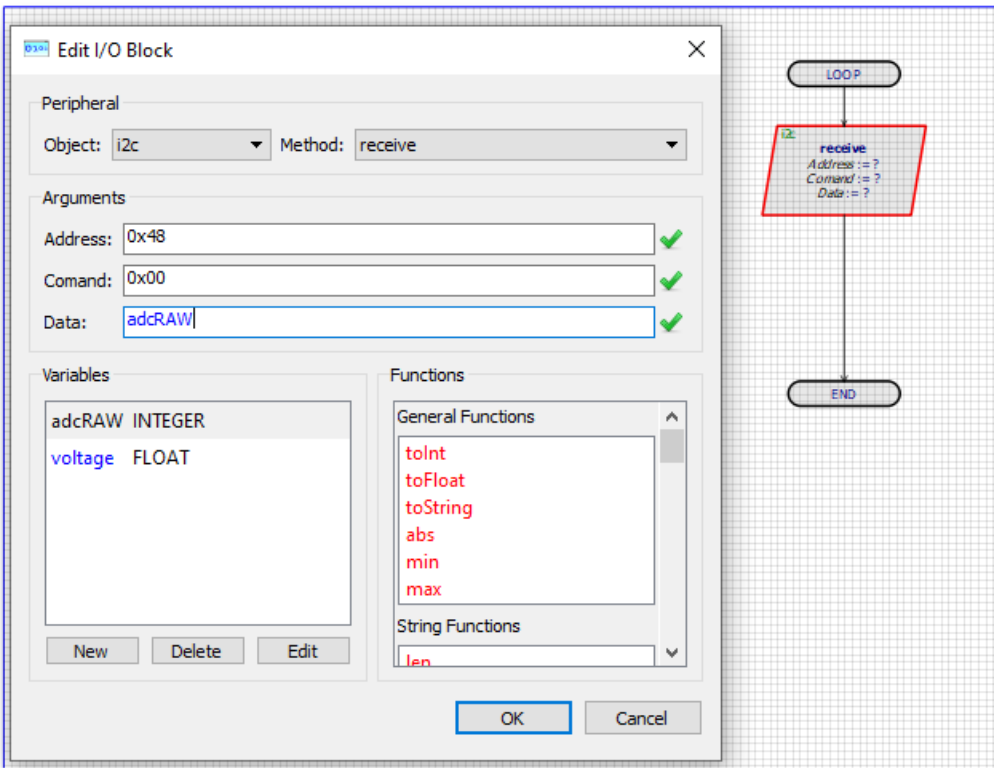


Fig 9.7 Add receive block and set values

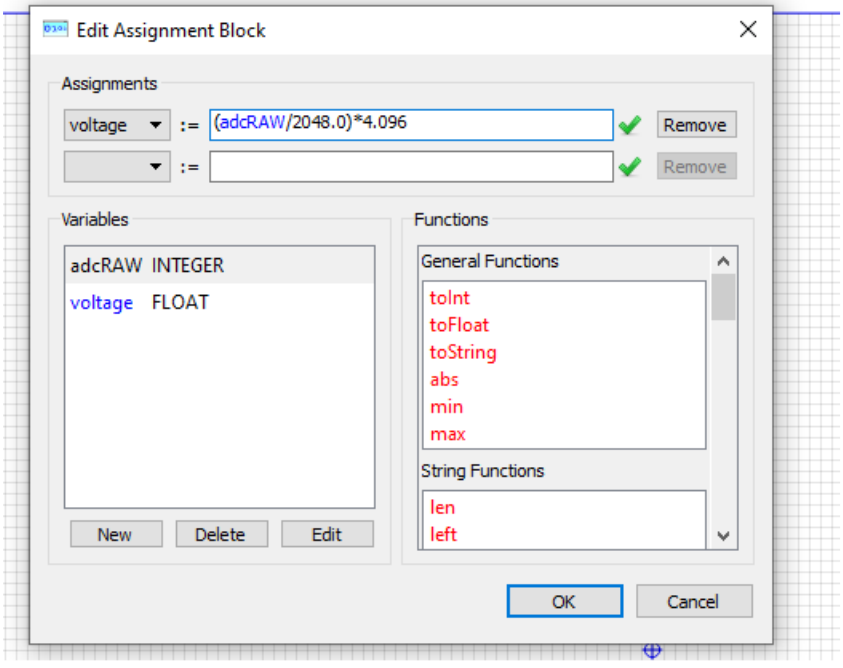
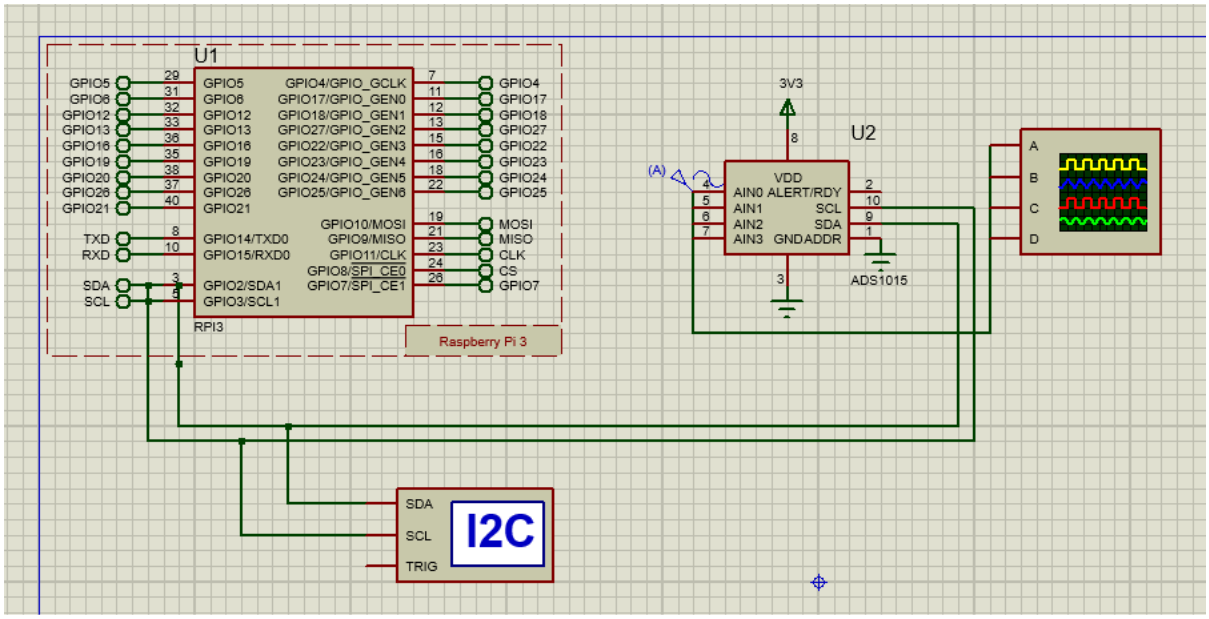


Fig 9.8 Add assignment block and assign values



Fig 9.9 see voltage



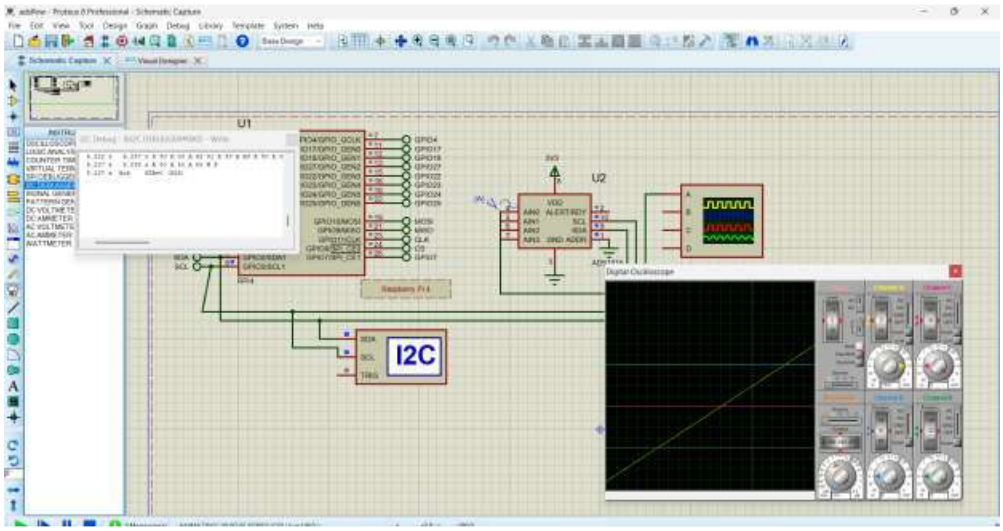


Fig 9.10 pick oscilloscope and i2c debugger and connect

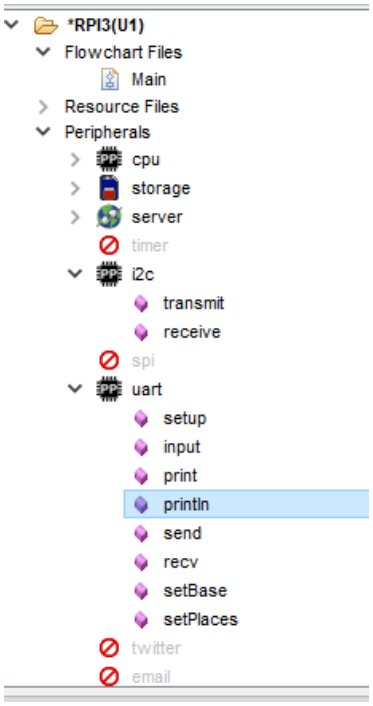


Fig 9.11 In Visual Designer's Peripherals list, right-click 'uart' (currently disabled/red) and

enable it

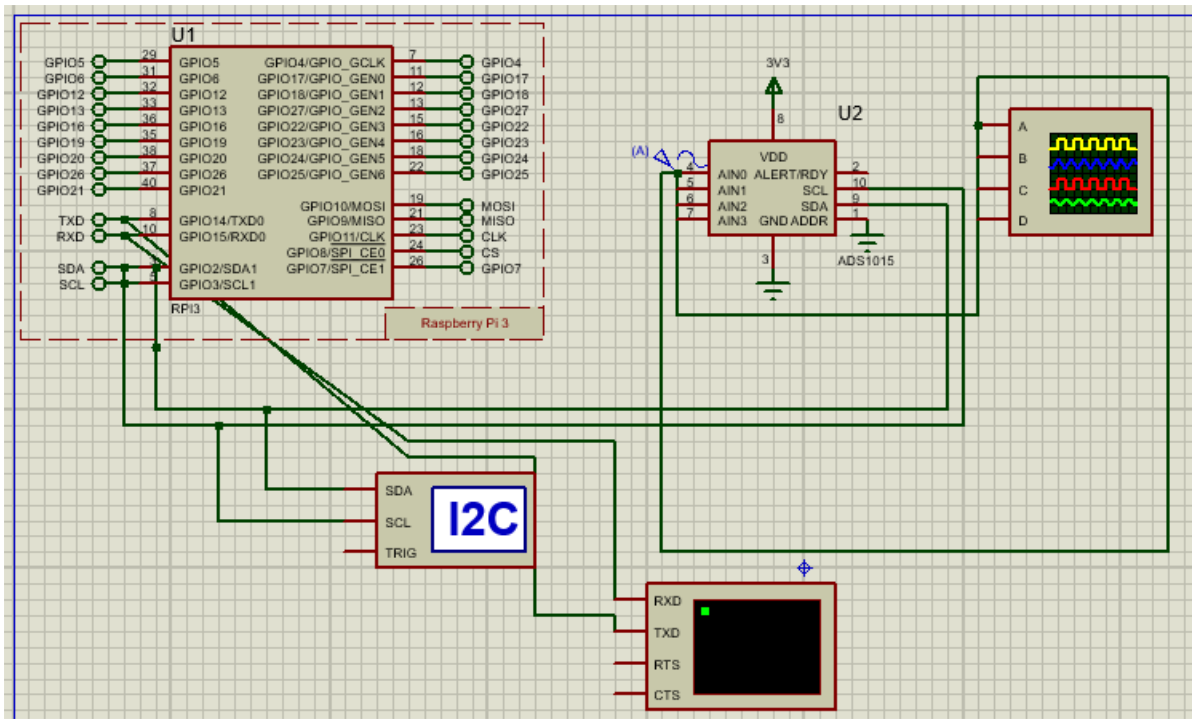


Fig 9.12 In Schematic Capture, open the Instruments panel (left side, same list where you

found OSCILLOSCOPE and I2C DEBUGGER). Select 'VIRTUAL TERMINAL' and

place it on the schematic near the Raspberry Pi.

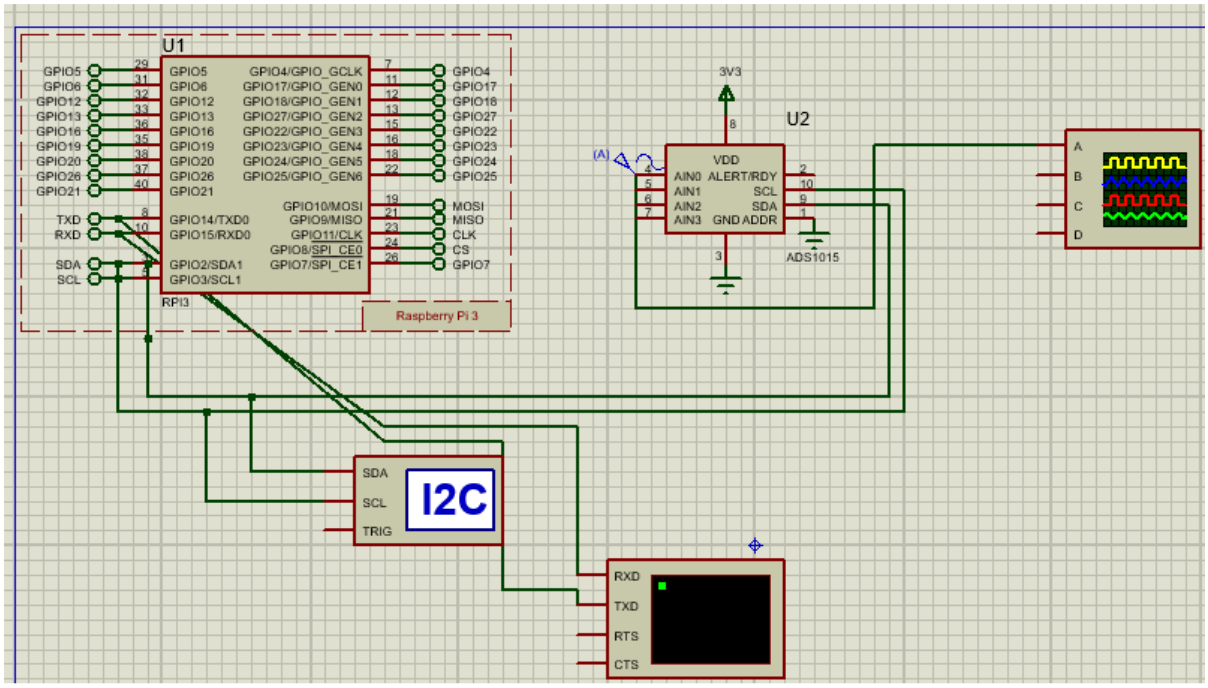


Fig 9.13 Wire the Pi's TXD (physical pin 8) to the Virtual Terminal's RXD pin, and the Pi's

RXD (physical pin 10) to the Virtual Terminal's TXD pin — TX always connects to the other device's RX, and vice versa.

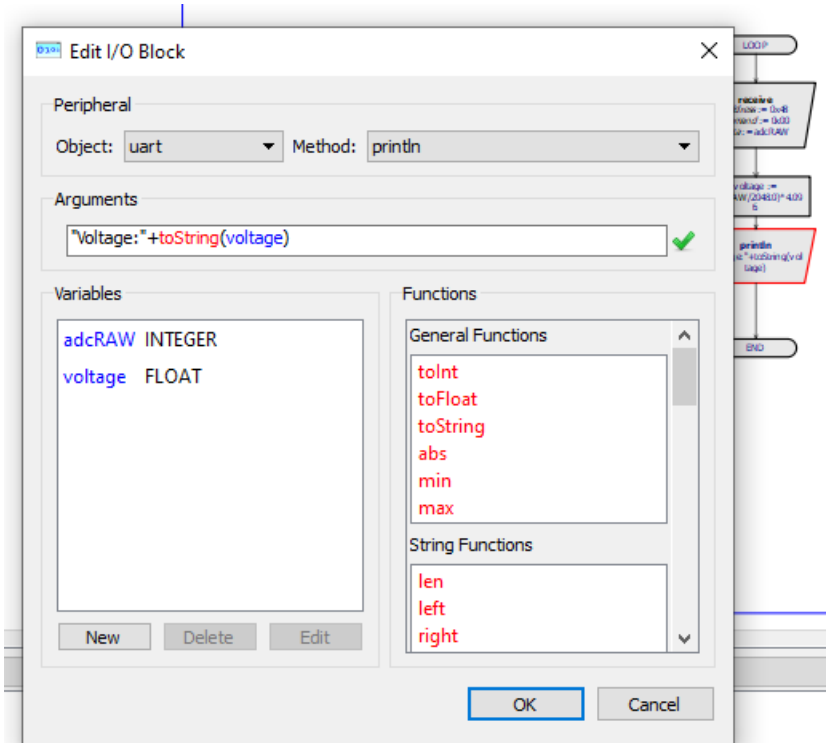


Fig 9.14 Drag 'println' from the uart peripheral list (left panel) onto the flowchart canvas,

dropping it on the arrow between your calculation block (voltage :=...) and END, inside the

LOOP branch.

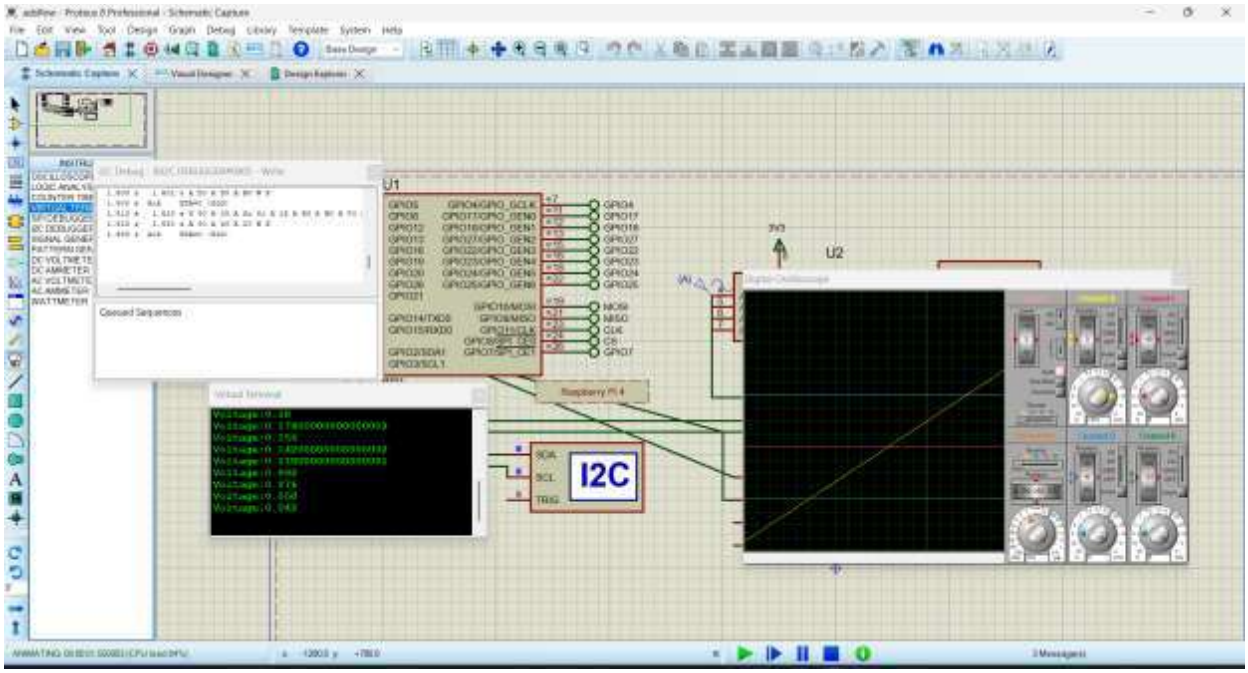
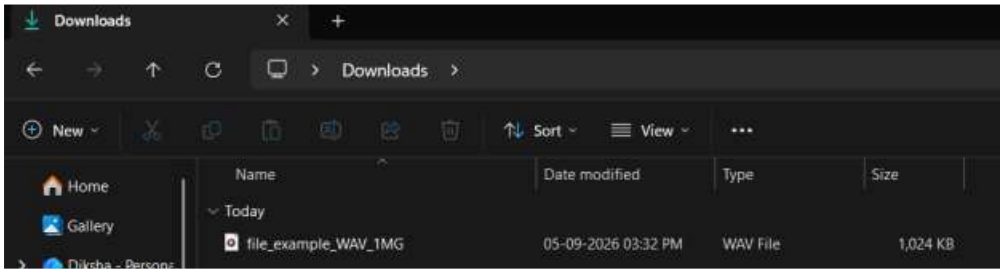


Fig 9.15 Execute the Simulation by pressing Run



Double-click your existing SINE generator (the one labeled 'U2(AIN0)' feeding into AIN0) to

reopen its Properties dialog.

Under 'Analogue Types', change the selection from 'Sine' to 'Audio'. The dialog will update to

show an audio file path field.

Click the browse/file button and select a short .wav file (mono, ideally low sample rate like 8-

16kHz keeps simulation fast). If you don't have one handy, any short spoken-word or tone .wav

clip works for testing.

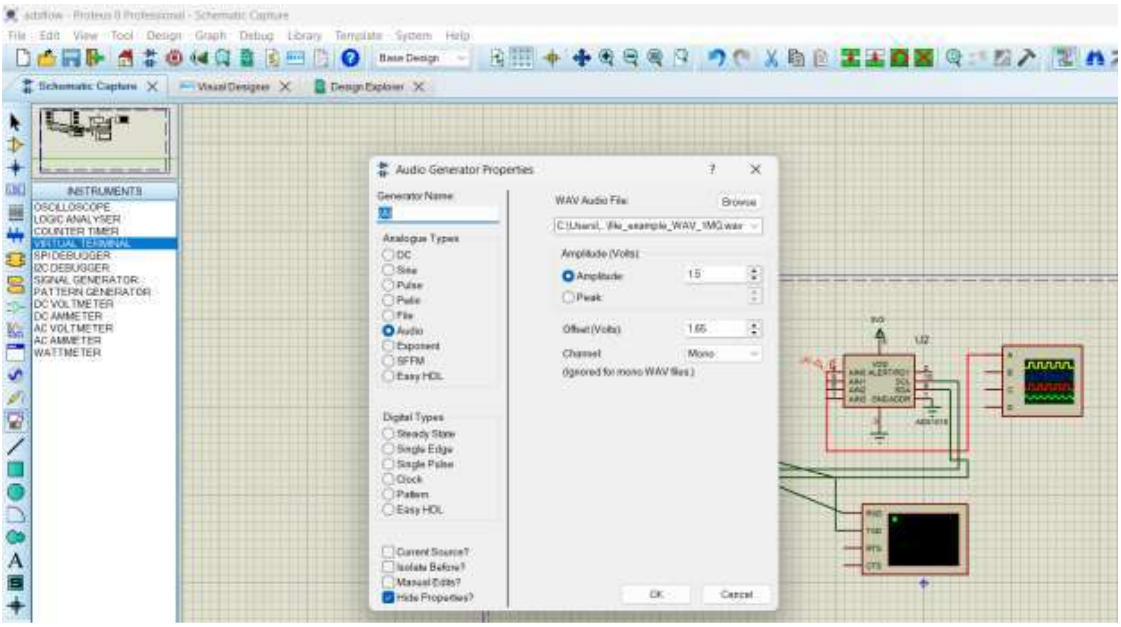


Fig 9.16 Provide Downloaded Audio to Audio Generator

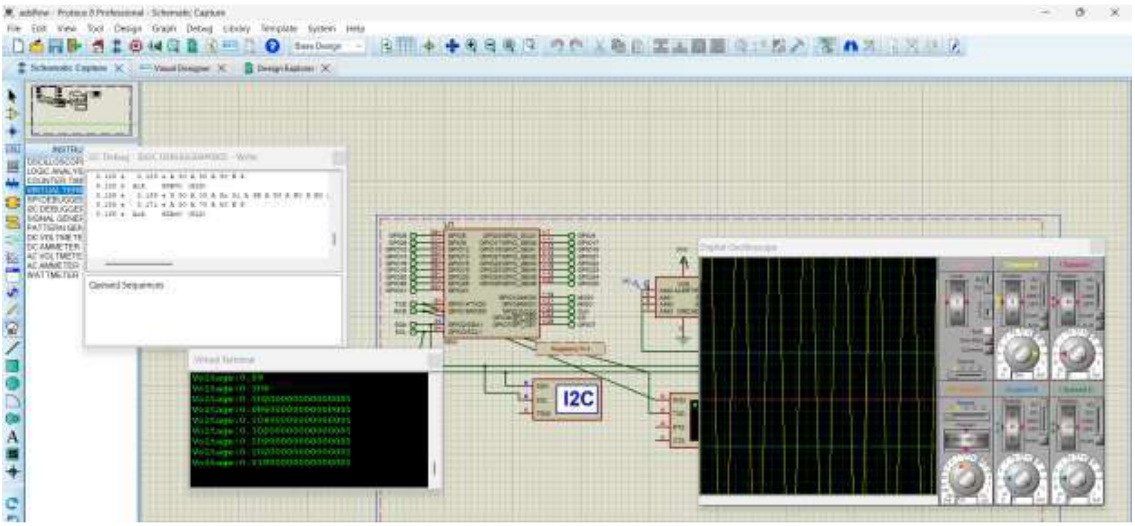


Fig 9.17 Final Execution on simulation